

UNIVERSITY  
OF TWENTE.



MSc Business Administration

University of Twente

MSc Innovation Management, Entrepreneurship & Sustainability

Technische Universität Berlin

---

# **Technical Debt as a Strategic Trade-Off: An Empirical Analysis of Execution Speed and Funding Success in Venture-Backed Startups**

---

## **Author**

Max Heichling (3696421)

## **Supervisors**

Dr. Maximilian Goethner (University of Twente)

Dr. Igors Skute (University of Twente)

February 3, 2026

*Für meinen Vater*

---

# Contents

---

|          |                                                                                |           |
|----------|--------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                            | <b>1</b>  |
| 1.1      | Background: The Startup Dilemma . . . . .                                      | 1         |
| 1.2      | Problem Statement . . . . .                                                    | 2         |
| 1.3      | Research Question . . . . .                                                    | 3         |
| 1.4      | Contribution of this Thesis . . . . .                                          | 3         |
| 1.5      | Thesis Outline . . . . .                                                       | 4         |
| <b>2</b> | <b>Literature Review</b>                                                       | <b>6</b>  |
| 2.1      | Defining and Measuring Technical Debt: The Traditional Engineering Perspective | 6         |
| 2.1.1    | The Debt Metaphor and Its Evolution . . . . .                                  | 6         |
| 2.1.2    | Types and Manifestations of Technical Debt . . . . .                           | 8         |
| 2.1.3    | Measurement Approaches and Strategic Context . . . . .                         | 8         |
| 2.2      | Development Velocity & Agility in Software Engineering . . . . .               | 10        |
| 2.2.1    | Conceptualizing and Measuring Velocity . . . . .                               | 10        |
| 2.2.2    | A Composite Velocity Metric for Startups . . . . .                             | 10        |
| 2.2.3    | Theorizing the Debt-Velocity Relationship . . . . .                            | 11        |
| 2.3      | Venture Capital and Startup Success Metrics: The ZIRP-Era Perspective . . . .  | 12        |
| 2.3.1    | Venture Capital Evaluation and the Primacy of Momentum . . . . .               | 13        |
| 2.3.2    | The ZIRP Era and the "Growth-at-All-Costs" Imperative . . . . .                | 13        |
| 2.3.3    | Operationalizing Venture Success . . . . .                                     | 14        |
| 2.4      | Synthesis, Research Gap, and Limitations . . . . .                             | 15        |
| <b>3</b> | <b>Methodology</b>                                                             | <b>17</b> |
| 3.1      | Research Design and Sample Construction . . . . .                              | 17        |
| 3.1.1    | Population Definition and Systematic Filtering . . . . .                       | 17        |
| 3.2      | Data Collection and Measurement Procedures . . . . .                           | 18        |
| 3.2.1    | Automated Analysis Pipeline . . . . .                                          | 19        |
| 3.2.2    | Static Analysis Tool Selection and Validation . . . . .                        | 19        |
| 3.2.3    | Technical Debt Ratio Implementation and Economic Modeling . . . . .            | 20        |
| 3.2.4    | Theoretical Justification for TDR as Strategic Proxy . . . . .                 | 21        |
| 3.2.5    | Composite Velocity Framework . . . . .                                         | 22        |
| 3.3      | Statistical Challenges and Solutions . . . . .                                 | 22        |
| 3.3.1    | Addressing the Nested Data Structure Problem . . . . .                         | 23        |
| 3.3.2    | Statistical Power and Inference Procedures . . . . .                           | 23        |

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| 3.4      | Strategic Framework and Analysis Approaches . . . . .                  | 24        |
| 3.4.1    | Primary Strategic Framework: The Debt-Velocity Matrix . . . . .        | 24        |
| 3.4.2    | Comprehensive Sensitivity Analysis Framework . . . . .                 | 25        |
| 3.5      | Validation and Quality Assurance . . . . .                             | 26        |
| 3.5.1    | Cross-Framework Validation . . . . .                                   | 26        |
| 3.5.2    | Outlier Management and Sensitivity . . . . .                           | 26        |
| 3.5.3    | Study Limitations and Mitigation Strategies . . . . .                  | 26        |
| <b>4</b> | <b>Results</b>                                                         | <b>28</b> |
| 4.1      | Sample Characteristics and Descriptive Analysis . . . . .              | 28        |
| 4.2      | Strategic Relationships: Debt, Velocity, and Funding Success . . . . . | 30        |
| 4.2.1    | The Debt-Velocity Relationship . . . . .                               | 30        |
| 4.3      | Strategic Framework and Robustness Analysis . . . . .                  | 31        |
| 4.3.1    | Robustness Testing Across Classification Schemes . . . . .             | 33        |
| 4.3.2    | Performance by Individual Metrics . . . . .                            | 34        |
| 4.4      | Synthesis of Key Findings . . . . .                                    | 35        |
| <b>5</b> | <b>Discussion</b>                                                      | <b>37</b> |
| 5.1      | What the Debt-Velocity Disconnect Really Means . . . . .               | 37        |
| 5.2      | The Strategic Debt Success Pattern . . . . .                           | 38        |
| 5.3      | Why Velocity Dominates in Venture Context . . . . .                    | 39        |
| 5.4      | Alternative Explanations and Competing Interpretations . . . . .       | 39        |
| 5.5      | Implications for Theory and Practice . . . . .                         | 40        |
| 5.5.1    | Theoretical Contributions . . . . .                                    | 40        |
| 5.6      | Implications for Theory and Practice . . . . .                         | 41        |
| 5.6.1    | Extending Technical Debt Theory . . . . .                              | 41        |
| 5.6.2    | Practical Implications . . . . .                                       | 42        |
| 5.7      | Study Boundaries and Future Research Directions . . . . .              | 43        |
| <b>6</b> | <b>Conclusion</b>                                                      | <b>44</b> |
|          | <b>List of Figures</b>                                                 | <b>46</b> |
|          | <b>List of Tables</b>                                                  | <b>47</b> |
|          | <b>References</b>                                                      | <b>48</b> |
| <b>A</b> | <b>Appendices</b>                                                      | <b>52</b> |
| A.0.1    | Data Collection Notes . . . . .                                        | 57        |

---

# List of Acronyms

---

|               |                           |
|---------------|---------------------------|
| <b>TDR</b>    | Technical Debt Ratio      |
| <b>DV</b>     | Development Velocity      |
| <b>VC</b>     | Venture Capital           |
| <b>ZIRP</b>   | Zero Interest-Rate Policy |
| <b>COCOMO</b> | Constructive Cost Model   |
| <b>IQR</b>    | interquartile range       |

## **Abstract**

The conventional wisdom in software engineering emphasizes the negative impact of technical debt on development productivity and long-term sustainability. However, this perspective may not adequately account for the strategic context of resource-constrained startups operating under venture capital pressure for rapid growth. This study empirically examines the relationship between technical debt accumulation, development velocity, and funding success in venture-backed software companies during the post-2008 low-interest-rate environment. Using a novel automated analysis pipeline, this research analyzed 70 open-source venture-backed companies across 146 funding periods. Technical debt was measured using the established Technical Debt Ratio (TDR) calculation methodology, while Development Velocity (DV) was assessed through a composite metric building upon the Cloud Native Computing Foundation's approach to measuring open source project velocity, incorporating code churn, commit frequency, and team engagement. Funding success was operationalized as the ability to secure subsequent funding rounds and funding growth rates. The analysis revealed that technical debt does not systematically constrain development velocity within typical startup ranges ( $r = 0.056$ ,  $p = 0.667$ , company-level), challenging conventional software engineering assumptions. This weak relationship explains why development velocity emerged as a substantially stronger predictor of funding success than technical debt levels. Notably, the highest-performing quadrant combines high technical debt with high velocity (60.6% success rate), while high-velocity development emerged as the critical factor across all debt levels, suggesting that strategic debt accumulation may actually enable faster execution in venture-backed environments. These findings challenge the universally negative characterization of technical debt in software engineering literature, proposing instead that its impact is context-dependent on macroeconomic conditions and organizational objectives. In capital-abundant environments, the market appears to reward demonstrated momentum over internal code quality, supporting a strategic framework where development velocity, rather than technical debt management, serves as the critical factor for competitive advantage.

# Introduction

---

## 1.1 Background: The Startup Dilemma

The modern software startup ecosystem embodies a fundamental tension between two competing engineering philosophies. On one side stands Facebook's notorious "move fast and break things" ethos, championing rapid iteration and accepting imperfect code as a necessary cost of market velocity. On the other, traditional software engineering wisdom advocates for the "slow down and build it right" principle, emphasizing careful design and sustainable development practices.

This philosophical divide becomes particularly acute when examining venture execution, defined here as a startup's ability to rapidly iterate and deliver software features at a pace that demonstrates momentum to investors (Gompers & Lerner, 2001; Lee & Kim, 2024), operationalized through development velocity and market validation via successful funding rounds. Development velocity represents the technical dimension of how rapidly a team can iterate and deploy features, while technical debt accumulation reflects the engineering trade-offs made in pursuit of such speed (Cunningham, 1992; Besker et al., 2018).

Technical debt, a metaphor introduced by Ward Cunningham to describe the implied cost of choosing an expedient solution instead of a better approach (Cunningham, 1992), manifests in shortcuts in code architecture, deferred refactoring, inadequate documentation, and accumulated complexity. While conventional software engineering literature universally characterizes technical debt as detrimental to long-term productivity, this perspective may not adequately account for the unique strategic context faced by resource-constrained startups.

Venture-backed software companies operate under extraordinary pressure for rapid growth and milestone achievement, with funding rounds serving as critical validation points for survival and continued capital access (Sahlman, 1990). Staged financing allows investors to mitigate risk by evaluating startups at discrete milestones, with each successful round signaling market traction and reducing information asymmetry (Lévesque, 2000; Gompers & Lerner, 2001). The post-2008 macroeconomic environment, characterized by the Federal Reserve's Zero Interest-Rate Policy (ZIRP), fundamentally altered this dynamic. During the ZIRP era (c. 2009-2022), near-zero interest rates flooded the startup ecosystem with abundant capital (Ning et al., 2015), creating a "growth-at-all-costs" environment where rapid execution often superseded operational

efficiency (Kuratko et al., 2020). This context was intensified by the compression of competitive windows: the average time between product introduction and competitive entry contracted from 33 years in the early 20th century to just 3.4 years for recent innovations (Agarwal & Gort, 2001), making first-mover advantages increasingly valuable (Lieberman & Montgomery, 1998). Under these conditions, aggressive technical debt accumulation became a potentially rational strategic choice—a form of temporal arbitrage where organizations trade future development efficiency for immediate execution capability when current capacity is demonstrably more valuable than equivalent future capacity (Besker et al., 2018).

## 1.2 Problem Statement

Despite the prevalence of technical debt discussions in both academic literature and industry discourse, there exists a significant gap in empirical understanding of its actual impact on venture execution and funding success, particularly within the unique macroeconomic context of the ZIRP era. Technical debt has been extensively studied within software engineering (Li et al., 2015), and startup development practices have been examined in entrepreneurship literature (Klotins et al., 2019; Nguyen-Duc et al., 2021; Besker et al., 2018). However, these research streams have largely evolved in parallel. Venture capital evaluation literature analyzes funding determinants (Gompers & Lerner, 2001; Lee & Kim, 2024) but treats technical execution as a black box. As illustrated in Figure 1.1, this thesis addresses the intersection of these domains, examining how technical debt accumulation and development velocity associate with funding success in the capital-abundant ZIRP era.

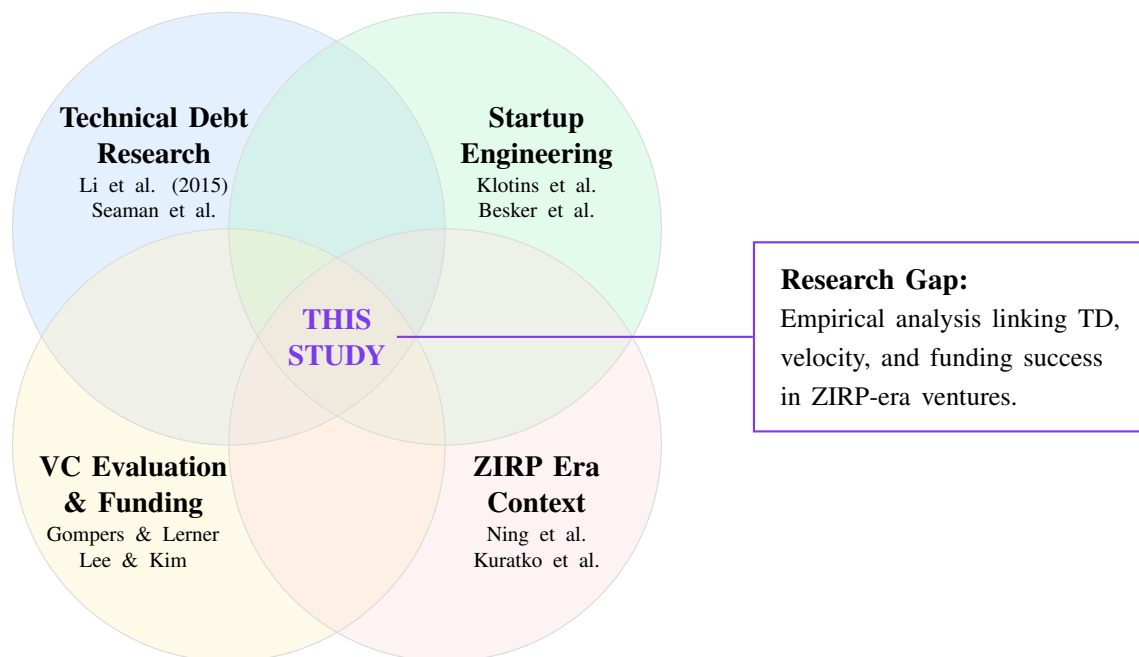


Figure 1.1 Research Gap at the Intersection of Technical Debt, Startup Strategy, VC Funding, and ZIRP-Era Context



While Besker et al. (2018) and Cico et al. (2021) have explored technical debt in startup contexts through qualitative interviews and case studies, no prior research has systematically quantified these relationships across multiple companies and funding cycles, nor examined how macroeconomic conditions moderate the strategic implications of engineering trade-offs. The existing research predominantly relies on anecdotal evidence or small-N case studies that may not adequately capture the complex trade-offs faced by venture-backed startups operating under capital abundance and time pressure.

This research gap is particularly problematic given the substantial resources allocated to technical debt management in the startup ecosystem and the potential strategic implications for both entrepreneurs and Venture Capital (VC) investors. Without empirical evidence of technical debt’s actual impact on business outcomes in the VC context, stakeholders lack the quantitative foundation necessary to make informed decisions about resource allocation between code quality maintenance and feature development velocity.

## 1.3 Research Question

This thesis addresses a central strategic question for venture-backed software companies:

**Research Question:** How do technical debt accumulation and development velocity, individually and in combination, associate with a startup’s ability to secure subsequent rounds of funding?

This question encompasses two analytical components that inform the strategic framework. First, examining whether technical debt systematically constrains development velocity tests the conventional assumption that debt creates “interest payments” limiting execution speed. Second, analyzing how debt-velocity combinations predict funding success reveals which strategic positions enable venture outcomes during the ZIRP era. The research design treats technical debt and development velocity as organizational choices that may interact to influence investor confidence and capital access.

## 1.4 Contribution of this Thesis

This research makes three primary contributions to the fields of software engineering and entrepreneurship:

**Methodological Innovation:** The creation of a novel, automated analysis pipeline capable of conducting longitudinal analysis of code quality and development velocity in startup environments. This pipeline enables systematic examination of technical debt accumulation patterns across multiple companies and funding periods, providing a scalable approach for future research in this domain.

**Scientific Reproducibility and Open Data:** This research establishes a replicable founda-

tion for quantitative startup studies by making the underlying dataset and automated analysis pipeline publicly available. By providing a verifiable benchmark through the longitudinal analysis of 70 open-source VC-backed companies, the study offers a standardized framework for the research community to validate these findings or adapt the methodology to analyze emerging post-ZIRP economic environments.

**Strategic Framework:** Proposing a strategic framework for understanding technical debt in an entrepreneurial context that acknowledges the legitimacy of debt accumulation as a rational choice under specific macroeconomic and competitive conditions. This framework challenges the universally negative characterization of technical debt in traditional software engineering literature by demonstrating context-dependent outcomes.

The research employs the established Technical Debt Ratio (TDR) calculation methodology for measuring code quality, while development velocity is assessed through a composite metric building upon the Cloud Native Computing Foundation’s approach to measuring open source project velocity, incorporating code churn, commit frequency, and team engagement patterns.

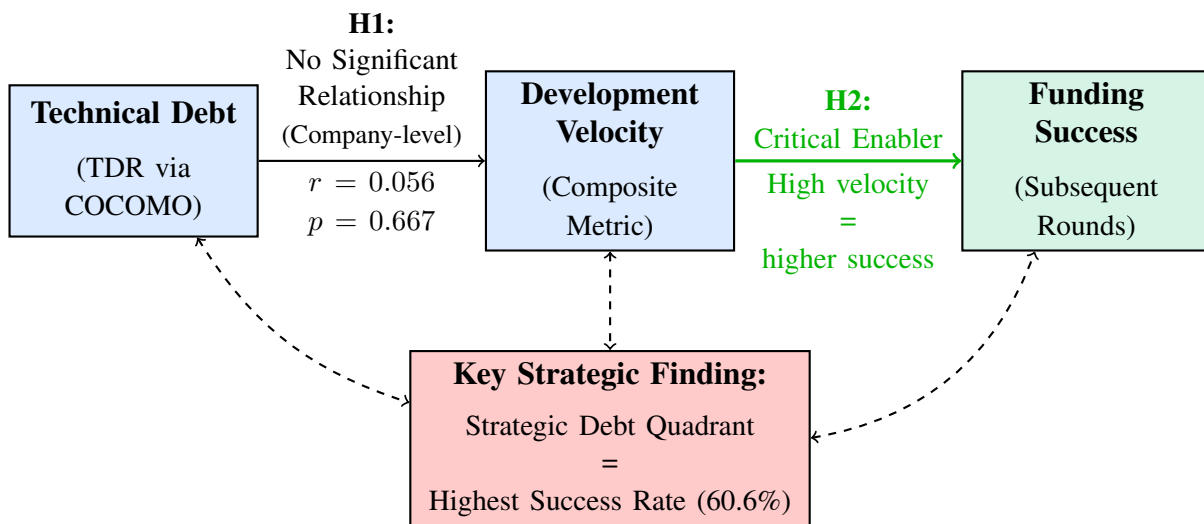


Figure 1.2 Conceptual Framework - Development Velocity as Primary Success Driver

## 1.5 Thesis Outline

The remainder of this thesis is structured as follows:

**Chapter 2: Literature Review** situates this work within existing academic and industry conversations, examining the theoretical foundations of technical debt measurement, development velocity assessment, and venture capital success metrics. This chapter establishes the conceptual framework underlying the empirical analysis and identifies specific gaps that this research addresses.

**Chapter 3: Methodology** provides a comprehensive account of the research design, data collection procedures, and analytical methods employed. This chapter details the automated analysis pipeline, including sample selection criteria, technical debt measurement using the Qlty platform, development velocity calculation methodology, and statistical analysis approaches.

**Chapter 4: Results** presents the empirical findings objectively, including the analysis of technical debt and development velocity relationships through both per-period and per-company approaches, performance analysis across strategic quadrants, and funding success patterns observed across the 146 funding periods analyzed.

**Chapter 5: Discussion** interprets the results within the broader context of software engineering and entrepreneurship literature, with particular attention to the implications of the non-significant correlation between technical debt and velocity, and the finding that high-velocity development emerged as a strong predictor of funding success, with the strategic debt quadrant (high debt + high velocity) achieving the highest success rate at 60.6%.

**Chapter 6: Conclusion** summarizes the key findings, acknowledges the limitations of the study, and proposes avenues for future research, including the need to examine whether these relationships hold in post-ZIRP economic environments characterized by capital scarcity and altered investor priorities.

# Literature Review

---

This chapter examines the theoretical foundations underlying the empirical analysis presented in this thesis, with particular attention to a fundamental tension that shapes modern startup strategy. The literature review is organized around three core domains: technical debt conceptualization and measurement, development velocity assessment in software engineering, and venture capital success metrics. However, these domains present conflicting strategic imperatives. Traditional software engineering principles advocate for rigorous technical debt management as essential for long-term sustainability, while the venture capital ecosystem—particularly during the zero interest-rate policy era—has explicitly rewarded rapid execution and market capture, often at the expense of code quality. A critical synthesis of recent and foundational literature from these fields establishes the conceptual framework necessary to investigate whether technical debt represents a strategic liability or a competitive tool in venture-backed software companies.

## 2.1 Defining and Measuring Technical Debt: The Traditional Engineering Perspective

The discourse surrounding technical debt provides the foundation for understanding the engineering perspective on quality versus speed trade-offs. This section examines the evolution of the technical debt metaphor, its various manifestations, and the prevailing methodologies for its measurement, while positioning these concepts within the traditional software engineering paradigm that views debt as fundamentally problematic.

### 2.1.1 The Debt Metaphor and Its Evolution

The concept of technical debt, first articulated by Ward Cunningham in 1992, has evolved from a simple metaphor for explaining design shortcuts to a comprehensive framework for understanding the long-term implications of software development decisions (Cunningham, 1992). Cunningham’s original formulation emphasized that shipping code that is not “quite right” is analogous to taking on financial debt. This initial act of “borrowing” against a clean, optimal design allows for immediate product delivery but incurs “interest payments” in the form

of increased development effort and complexity in all future work until the “principal” is repaid through refactoring. The metaphor’s power lies in its ability to frame a technical issue in clear business terms, suggesting that not all debt is inherently negative; it can be a strategic tool for leveraging market opportunities (Seaman et al., 2012). This strategic dimension becomes critical when startups face time-sensitive competitive windows. In fast-moving markets, the opportunity cost of delayed market entry can far exceed the interest payments on technical debt (Curtis et al., 2012). The debt metaphor thus captures a fundamental startup trade-off: accepting future maintenance costs (interest) to accelerate feature delivery and secure market position today. This calculus intensifies in venture-backed software companies, where securing subsequent funding rounds often depends on demonstrating rapid user growth and market traction within narrow windows (Lee & Kim, 2024). The metaphor therefore extends beyond software engineering to encompass the broader strategic question of resource allocation under conditions of capital scarcity and competitive pressure.

Martin Fowler’s influential “Technical Debt Quadrant” provides a more nuanced taxonomy, distinguishing between different types of technical debt based on whether it is deliberate or inadvertent, and prudent or reckless (Fowler, 2009). Deliberate and prudent debt represents a conscious, strategic decision to prioritize speed over perfection to meet a critical deadline, with a full understanding of the future costs. This contrasts with reckless and inadvertent debt, which arises from poor practices or lack of awareness and offers no strategic upside. Importantly, even within this framework, the traditional engineering perspective maintains that debt should be temporary and strategically managed—a view that, as we will explore, may be challenged by the unique pressures of venture-backed startups during periods of abundant capital.

|             | Reckless                               | Prudent                                              |
|-------------|----------------------------------------|------------------------------------------------------|
| Deliberate  | <i>“We don’t have time for design”</i> | <i>“We must ship now and deal with consequences”</i> |
| Inadvertent | <i>“What’s layering?”</i>              | <i>“Now we know how we should have done it”</i>      |

Figure 2.1 Fowler’s Technical Debt Quadrant

### 2.1.2 Types and Manifestations of Technical Debt

Technical debt is not a monolithic concept; it manifests in numerous forms throughout the software development lifecycle. While code debt is the most commonly measured, other forms can have equally significant impacts on a startup’s trajectory. A synthesized taxonomy based on recent literature includes (Morgenthaler et al., 2012):

**Code Debt** encompasses violations of coding standards, high cyclomatic complexity, duplicated code, and other “code smells” that degrade readability and maintainability (Arcelli Fontana et al., 2012). This represents the most immediately visible form of technical debt, readily detected through automated analysis tools.

**Architectural Debt** involves suboptimal design decisions at the system level that constrain future scalability and modification. This form of debt is often the most costly to repay, as it may require fundamental restructuring of system components.

**Test Debt** manifests as inadequate or missing test suites, leading to increased risk of bugs and higher cost of validation for new features. In startup environments, test debt is frequently accumulated due to time pressures for rapid deployment.

**Infrastructure and Build Debt** includes non-automated deployment processes, inconsistent development environments, and brittle build systems, all of which introduce friction into the development cycle and slow down velocity (Morgenthaler et al., 2012).

The primary causes of technical debt align closely with pressures inherent in startup environments. Time pressure to meet market windows or investor milestones represents the most cited cause (Li et al., 2015). This is compounded by business pressure for new features, volatility in requirements, and resource constraints that limit the availability of skilled personnel or time for proper refactoring. These causes highlight the fundamental tension: the very conditions that define startup success—rapid iteration, resource constraints, and urgent market pressures—are precisely those that the traditional engineering perspective identifies as debt-inducing.

### 2.1.3 Measurement Approaches and Strategic Context

The automated measurement of technical debt relies fundamentally on static code analysis—the examination of source code without executing it. This approach provides the scalability necessary for large-scale empirical research while offering objective, repeatable measurements of code quality. However, the relationship between static analysis outputs and actual strategic impact in venture-backed environments requires careful consideration, as the strategic calculus during the ZIRP era fundamentally altered traditional quality-cost trade-offs.

#### Static Analysis Foundations

The theoretical justification for using static analysis as a proxy for technical debt stems directly from Cunningham’s original debt metaphor (Cunningham, 1992). Static analysis tools sys-

tematically identify patterns in source code that correlate with poor maintainability, increased complexity, and higher likelihood of defects—precisely the characteristics that Cunningham described as incurring “interest payments” on technical debt.

However, static analysis primarily captures surface-level quality indicators that may not reflect the strategic debt accumulation patterns most relevant to startup velocity and funding outcomes. Architectural debt, arising from suboptimal system-level design decisions, often represents the most costly form of technical debt to remediate (Martini et al., 2014), yet remains largely invisible to conventional static analysis tools.

### **Economic Modeling and Strategic Context**

The economic framing of technical debt becomes particularly relevant in venture-backed environments, where engineering decisions are explicitly evaluated against financial constraints and investor expectations. The ability to quantify technical debt in monetary terms—time costs, development effort, delayed features—enables direct comparison with the capital abundance and growth pressures that define startup strategic contexts.

During the ZIRP era, abundant capital fundamentally altered the strategic calculus surrounding quality trade-offs. Traditional software engineering assumes that technical debt’s “interest payments” represent unavoidable future costs that rational actors should minimize. However, when external funding can continuously subsidize these interest payments, and when the penalty for slow market execution exceeds the long-term costs of debt accumulation, the strategic optimal may involve accepting higher debt levels in exchange for accelerated development velocity.

### **Strategic Implications for Venture-Backed Development**

This measurement constraint has profound implications for understanding technical debt’s strategic role in venture execution. Startups classified as “high debt” based on static analysis may actually maintain clean architecture and robust testing practices—the forms of technical health most crucial for sustaining rapid development velocity. Conversely, companies with excellent code-level metrics may harbor architectural decisions that fundamentally constrain their ability to scale, particularly relevant given the “blitzscaling” imperative during the ZIRP era.

The challenge extends beyond tool limitations to fundamental questions about what constitutes “strategic debt” in venture-backed environments. Traditional software engineering frameworks assume that all forms of technical debt eventually impose velocity constraints, but this assumption may not hold when external capital can continuously subsidize the costs of code-level inefficiencies while market timing pressures make architectural perfection a secondary concern. Nevertheless, code-level debt represents the most objectively measurable and comparable form of technical debt across organizations, providing a standardized foundation for empirical analysis. The specific implementation details and validation approaches for measuring technical debt in this research context are detailed in subsection 3.2.1, where the automated analysis pipeline

and tool selection criteria are comprehensively addressed.

## **2.2 Development Velocity & Agility in Software Engineering**

Development velocity became a central concept with the rise of agile methodologies, which prioritize delivery speed and iterative development over the rigid, plan-driven approaches of traditional software engineering (Beck et al., 2001). However, the measurement and interpretation of velocity takes on fundamentally different meanings when considered through the lens of venture capital evaluation criteria, as will be explored in the following section.

### **2.2.1 Conceptualizing and Measuring Velocity**

Within agile frameworks like Scrum, velocity is traditionally the number of “story points” a team completes per sprint (Sutherland, 2004). While useful for a single team’s internal planning, story points are subjective and not comparable across different organizations, limiting their utility for external research (Cohn, 2005). Consequently, researchers have turned to objective, repository-derived data to create standardized velocity metrics. This approach, exemplified by the Cloud Native Computing Foundation’s (CNCF) “project velocity” metric, synthesizes multiple activity indicators to provide a holistic view of project momentum (Cloud Native Computing Foundation, 2022). Recent work by Lee & Kim (2024) confirms that such objective measures, derived from job postings and development activity, are strong indicators of a startup’s intent to scale and its subsequent performance (Lee & Kim, 2024).

### **2.2.2 A Composite Velocity Metric for Startups**

For the specific context of startups, a velocity metric must capture not just raw output but the *speed of execution* that is critical for market validation and investor confidence. This composite velocity metric is designed specifically to quantify the “demonstrated momentum” that venture capital investors prioritize when evaluating startups for subsequent funding rounds, as detailed in subsection 2.3.1. The composite velocity metric developed for this study combines two complementary dimensions, normalized by team size to enable cross-organizational comparison. Code churn per author captures the volume of development activity per contributor, serving as a proxy for individual productivity and feature development intensity. Commits per author reflects the team’s development rhythm normalized by team size, with higher rates indicating more frequent iteration cycles and active development practices. By normalizing both metrics by author count and then by their respective maximum values across the dataset, the metric provides a standardized measure of a startup’s execution intensity that accounts for both team size and relative performance, making it suitable for the cross-organizational analysis undertaken in this thesis.



The metric incorporates two core dimensions:

**Code Churn per Author:** The volume of code being changed (lines added, deleted, modified) divided by the number of contributing authors. This normalizes raw development output by team size, providing a measure of per-capita development intensity.

**Commits per Author:** The number of commits over time divided by the number of contributing authors. This serves as an indicator of development rhythm and iteration frequency per contributor, accounting for team size variations across startups.

The composite velocity metric is calculated as:

$$\text{Composite Velocity} = \frac{1}{2} \left( \frac{\text{Code Churn per Author}}{\text{Max Code Churn per Author}} + \frac{\text{Commits per Author}}{\text{Max Commits per Author}} \right) \quad (2.1)$$

Where:

$$\begin{aligned} \text{Code Churn per Author} &= \frac{|\text{Lines Added} + \text{Lines Deleted}|}{\text{Period Days} \times \text{Author Count}} \\ \text{Commits per Author} &= \frac{\text{Total Commits}}{\text{Period Days} \times \text{Author Count}} \end{aligned}$$

This formulation normalizes both component metrics by their respective maximum values observed across the entire dataset, ensuring that the composite velocity ranges from 0 to 1 and enables meaningful comparison across startups with different team sizes and absolute activity levels. The equal weighting (0.5 each) reflects the complementary nature of code churn and commit frequency as indicators of development intensity, while the per-author normalization accounts for the significant variation in team sizes across venture-backed startups.

Additionally, the per-author normalization assumes that development contributions are roughly equivalent across team members, which may not hold in practice where senior developers contribute disproportionately or where non-development contributors (such as product managers) are included in author counts.

### 2.2.3 Theorizing the Debt-Velocity Relationship

The anticipated negative relationship between technical debt and development velocity builds on Cunningham’s original formulation that debt incurs "interest payments" in the form of increased development effort (Cunningham, 1992). This suggests three mechanisms through which debt impedes velocity. First, as documented by Li et al. (2015), technical debt increases code complexity, making it harder to understand and modify existing systems. Second, accumulated debt increases the likelihood of introducing defects during development, requiring additional debugging time. Third, architectural debt specifically can constrain implementation options, forcing developers to work around existing limitations rather than implementing

Table 2.1: Velocity Measurement Approaches for Cross-Company Startup Analysis

| Approach                  | Key Metrics                                 | Advantages                                                             | Limitations for Startup Research                                                                       |
|---------------------------|---------------------------------------------|------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Traditional Agile</b>  | Story points per sprint, burn-down rates    | Team-specific accuracy, established practice                           | Subjective scaling; not comparable across organizations                                                |
| <b>Repository Metrics</b> | Commits, LOC changed, PR frequency          | Objective, automated collection                                        | Activity-focused; may not reflect business value or feature completion                                 |
| <b>DORA Metrics</b>       | Deployment frequency, lead time, MTTR       | Operations-focused, industry standard                                  | Emphasizes stability over development speed; limited applicability to early-stage products             |
| <b>Composite Velocity</b> | Code churn + commits per author, normalized | Combines activity and team size; designed for cross-company comparison | Assumes uniform developer contributions; code activity may not correlate with market-relevant progress |

straightforward solutions (Morgenthaler et al., 2012).

However, while engineering principles suggest that accumulated debt will eventually impose "interest payments" that slow velocity, the strategic imperative during the ZIRP era often forced founders to consciously accept this future cost in exchange for the immediate survival and market validation that high velocity could unlock. This relationship may not be strictly linear, particularly in early-stage startups with relatively small codebases, where the absolute volume of debt may remain manageable enough that its impact on velocity is minimal. Additionally, Fowler's distinction between deliberate-prudent and reckless-inadvertent debt suggests that some forms of debt may actually enable higher short-term velocity by allowing teams to defer non-critical improvements (Fowler, 2009). This implies a nuanced relationship where the impact of technical debt on velocity depends on both the volume and type of debt accumulated—and, critically, on the external pressures and incentives that startups face in competitive markets.

## 2.3 Venture Capital and Startup Success Metrics: The ZIRP-Era Perspective

The VC financing model fundamentally shapes the strategic context for startups, often in direct opposition to traditional engineering wisdom about sustainable development practices. This section examines the evaluation criteria used by VCs, the influence of recent macroeconomic conditions, and the justification for using funding progression as a primary success metric.

### **2.3.1 Venture Capital Evaluation and the Primacy of Momentum**

VC investors evaluate startups across multiple dimensions, including team quality, market size, and business model viability (Gompers & Lerner, 2001). However, in software ventures specifically, demonstrated momentum or "traction" emerges as a particularly critical factor in subsequent funding decisions. Unlike capital-intensive sectors requiring substantial physical infrastructure, software ventures can translate development activity more directly into market capture through iterative deployment. In this context, a startup's development velocity becomes a powerful observable proxy for organizational capability, signaling its ability to execute in time-sensitive competitive environments (Lee & Kim, 2024).

This evaluative emphasis stems from information asymmetry. VCs typically lack the technical expertise or time to audit a startup's codebase architecture or accumulated technical debt (Lévesque, 2000). They rely instead on observable proxies: a startup's ability to consistently ship features that drive user growth becomes a composite signal of technical competence, team coordination, and market readiness. In software markets, where development cycles are compressed and iterative deployment is technically feasible, rapid execution often determines whether ventures secure market position before competitors emerge (Lévesque, 2000). The result is a strategic environment where execution speed is rewarded by capital markets, even when such speed may compromise code quality or long-term sustainability.

This prioritization varies across industries. Software products, as intangible assets, are "better suited for blitzscaling because they can be launched quickly and updated with relative ease," whereas sectors like pharmaceuticals or hospitals face regulatory constraints that make rapid iteration infeasible (Kuratko et al., 2020). The fundamental tradeoff facing software entrepreneurs is temporal: "entry too early may push an underdeveloped product into the marketplace. However, if entry is delayed too long, the firm may sacrifice the benefits of being first with a new product or technology" (Lévesque, 2000). Ventures operating under compressed competitive windows thus face pressure to establish market presence with minimum viable products, accepting technical imperfection in exchange for market timing. This dynamic creates a fundamental tension between the market's preference for observable momentum and engineering's emphasis on sustainable, maintainable code—a tension that shapes technical debt accumulation patterns in venture-backed software companies.

### **2.3.2 The ZIRP Era and the "Growth-at-All-Costs" Imperative**

The strategic calculus of venture-backed startups from roughly 2009 to 2022 was profoundly shaped by the macroeconomic environment of the ZIRP era. This period of historically low interest rates led to an unprecedented influx of capital into the venture ecosystem, creating intense competition among investors and an abundance of "dry powder" available for startups (Ning et al., 2015; Kuratko et al., 2020).

This capital abundance fostered a "growth-at-all-costs" or "blitzscaling" investment strategy that fundamentally challenged traditional engineering approaches to sustainable development (Kuratko et al., 2020). With capital readily available, the primary competitive differentiator became speed of execution and market capture rather than technical excellence. The strategic imperative shifted from capital efficiency to capturing market share as rapidly as possible, often by subsidizing growth through massive funding rounds (Lee & Kim, 2024). In this environment, the penalty for being slow to market was perceived as far greater than the long-term costs of accumulating technical debt—a calculation that directly inverts traditional software engineering priorities.

However, this "growth-at-all-costs" paradigm was not without its critics even during the ZIRP era. Sustainable growth advocates argued that the emphasis on velocity over sustainability created fragile companies vulnerable to market corrections, as evidenced by the spectacular failures of WeWork and other high-growth ventures that prioritized expansion over unit economics (Kuratko et al., 2020). The tension between these perspectives—aggressive growth versus sustainable development—mirrors the technical debt debate at the engineering level, suggesting that optimal strategies may be contingent on both macroeconomic conditions and company-specific factors such as competitive dynamics and market maturity.

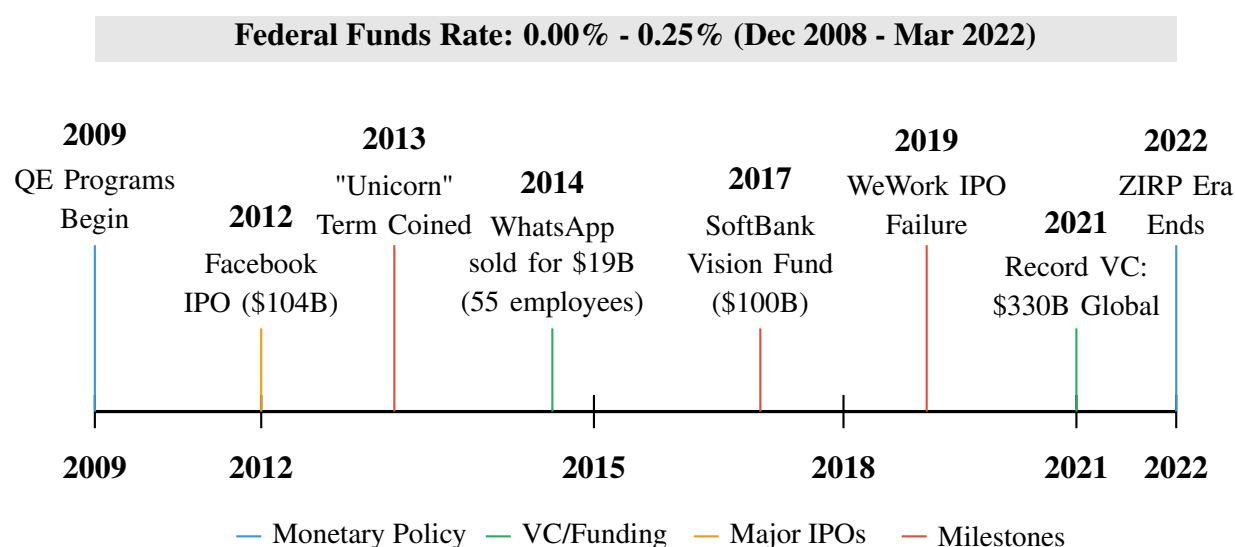


Figure 2.2 Timeline of the Zero Interest-Rate Policy Era (2009-2022)

### 2.3.3 Operationalizing Venture Success

The staged nature of venture capital provides a natural framework for measuring startup performance. Securing a subsequent funding round is a clear, binary indicator of success, signifying that a company has met the milestones required to attract further investment. Beyond this binary outcome, the funding growth rate—the percentage increase in capital raised between rounds—serves as a continuous measure of investor confidence and perceived value creation.

These funding-based metrics are well-established proxies for startup success in the venture ecosystem (Gompers & Lerner, 2001). They represent the revealed preferences of sophisticated investors who have conducted due diligence and are "voting with their wallets." Unlike more subjective measures, funding outcomes are objective, quantifiable, and directly reflect the market's assessment of a startup's progress and future potential. Critically, these metrics capture the market's implicit valuation of the speed-versus-quality trade-offs that startups make—providing an external validation mechanism for strategies that prioritize development velocity over traditional engineering best practices.

## 2.4 Synthesis, Research Gap, and Limitations

The literature from software engineering and venture capital presents a direct and fundamental conflict regarding the strategic management of technical debt in startups. Software engineering research extensively documents the negative long-term impacts of technical debt, framing it as a liability to be rigorously managed for sustainable development (Colazo, 2010; Arcelli Fontana et al., 2012). This traditional engineering perspective emphasizes that debt accumulation inevitably leads to decreased velocity, increased maintenance costs, and systemic fragility that threatens long-term viability.

The strategic calculus surrounding technical debt varies depending on financing structure. Debt-financed ventures face fixed repayment obligations that prioritize cash flow stability, while bootstrapped companies retain autonomy to optimize for sustainability (Lévesque, 2000). This thesis focuses on venture capital financing because it creates extreme time pressure: VC funds operate on finite horizons requiring portfolio companies to achieve substantial scale within compressed timeframes, intensifying the velocity imperative beyond other funding mechanisms (Gompers & Lerner, 2001).

In sharp contrast to engineering wisdom, the prevailing narrative in entrepreneurship and VC literature, particularly from the ZIRP era, emphasizes that for equity-backed companies, speed, momentum, and market traction are the primary determinants of survival and access to capital (Kuratko et al., 2020). This temporal constraint creates explicit incentives for rapid execution that directly oppose traditional engineering wisdom about sustainable development practices. The costs of being slow to market in VC-backed ventures—missed competitive windows, diminished investor interest, lost first-mover advantages—are perceived to far outweigh the future costs of technical debt accumulation. The VC ecosystem's explicit reward structure thus creates a strategic environment fundamentally different from debt-financed or bootstrapped alternatives.

This tension reveals a critical conceptual gap: established engineering models for quality management fail to account for the unique incentive structures, competitive dynamics, and survival pressures facing equity-backed ventures operating under compressed exit timelines. The tradi-

tional debt metaphor, which assumes that "interest payments" will eventually constrain growth, may not hold when external capital can continuously subsidize the costs of poor code quality, and when market timing constraints make speed existentially important.

A significant empirical research gap exists at this intersection. There is a lack of large-scale, quantitative research examining the relationship between technical debt, development velocity, and funding outcomes for startups during periods of capital abundance. Most technical debt studies focus on mature organizations with stable revenue streams, or rely on methodologies not suited for cross-company comparison (Li et al., 2015). This has left the strategic debate dominated by anecdote and theoretical frameworks rather than systematic empirical evidence about how these trade-offs actually play out in practice.

This thesis directly addresses this gap by providing a large-scale, quantitative analysis of the relationship between objectively measured technical debt, development velocity, and funding success during the capital-abundant ZIRP era. The research design faces inherent limitations, including sample selection constraints, measurement validity concerns, and temporal specificity, which are addressed in detail in the methodology (Section 3) and discussed with respect to generalizability and future research directions in Section 5.7.

Despite these limitations, this study provides a critical empirical foundation for moving beyond the theoretical tensions between engineering and venture perspectives toward a more nuanced, context-aware understanding of technical debt's strategic role in venture execution. The research aims to determine whether the traditional engineering view of debt as liability holds under the unique conditions of venture-backed growth, or whether debt might indeed represent a strategic tool for market capture during periods of abundant capital.

## Methodology

This chapter details the empirical methodology employed to investigate the relationship between technical debt, development velocity, and funding success in venture-backed software companies during the ZIRP era. The approach combines automated large-scale code analysis with longitudinal funding data to provide a systematic, cross-company examination of these relationships in the startup ecosystem.

### 3.1 Research Design and Sample Construction

This study employs a quantitative, longitudinal design examining 70 venture-backed companies across 146 inter-funding periods between 2009-2022. The unit of analysis is the development period between consecutive funding rounds, enabling observation of how technical debt accumulation and development velocity patterns associate with subsequent funding success. This design addresses a fundamental limitation in existing technical debt research, which has predominantly relied on single-company case studies or cross-sectional analyses of mature organizations Li et al. (2015).

The longitudinal approach captures the dynamic strategic trade-offs that characterize venture-backed development. By analyzing the same companies across multiple funding cycles, the design helps isolate the relationship between technical execution patterns and market validation through investor decisions.

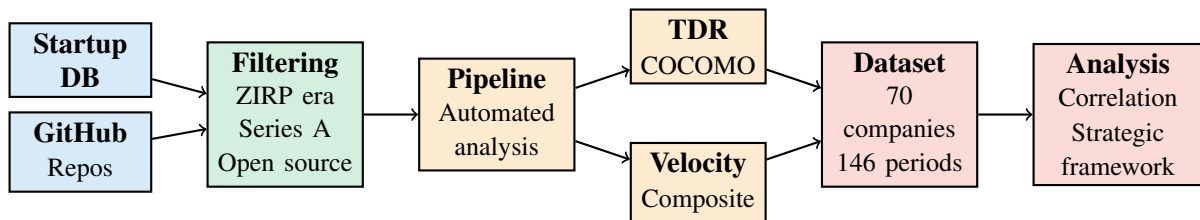


Figure 3.1 Research Methodology Overview

#### 3.1.1 Population Definition and Systematic Filtering

Sample construction began with an initial population of venture capital-backed software companies identified through comprehensive database searches of Crunchbase and Tracxn. This

population underwent systematic filtering to ensure analytical rigor and theoretical relevance. The temporal filtering criterion required companies to have raised Series A funding during the ZIRP era (2009-2022). Data suggests a typical timeframe of 12-24 months between funding rounds for venture-backed companies Gornall & Strebulaev (2020). To ensure sufficient time for a subsequent round to be evaluated, a cutoff was established requiring at least 36 months between the Series A date and the data collection point. Series A represents a critical validation milestone where companies are expected to have demonstrated product-market fit beyond initial seed-stage experiments, making it an appropriate baseline for examining development practices under venture pressure Sahlman (1990).

All funding information was cross-validated using Crunchbase and Tracxn databases. The open-source requirement, while a limitation on generalizability, was necessary to enable the comprehensive code analysis for technical debt measurement. Companies were included only when their primary product codebase was publicly accessible.

The remaining companies were classified into six industry categories:

Table 3.1: Distribution of Companies by Industry Category

| Industry Category               | Count |
|---------------------------------|-------|
| Developer Tools & DevOps        | 23    |
| Database & Storage              | 18    |
| Data Platform & Analytics       | 12    |
| Business & Application Software | 8     |
| Security                        | 5     |
| AI/Machine Learning             | 4     |

Further technical filtering was applied to ensure the quality of the data. A minimum threshold of 5,000 average lines of code across observation periods was required to ensure that TDR calculations reflect substantial software systems rather than early prototypes. Additionally, development periods shorter than 90 days were removed to avoid noise from insignificant development activity.

This comprehensive filtering process reduced an initial set of 153 potential development periods to a final analytical sample of 146 periods from 70 companies, yielding a data retention rate of 95.4%. This process balances the need for a sufficiently large sample with the requirements of data quality for robust analysis.

## 3.2 Data Collection and Measurement Procedures

This section details the automated analysis pipeline and the specific methodologies for measuring technical debt and development velocity. The approach combines novel automation techniques with established measurement frameworks to enable large-scale, objective analysis.



### 3.2.1 Automated Analysis Pipeline

A core component of this research is a novel, automated pipeline that synchronizes funding timeline data with historical code repository states. This pipeline addresses the challenge of measuring technical debt as it existed at the point of investor decision-making.

For each company, the pipeline clones the primary product repository. To align with the venture capital due diligence process, which evaluates a company’s state leading up to an investment decision Sahlman (1990), the analysis pipeline checks out the repository to the commit state closest to, but not after, each funding announcement date. This ensures that technical debt measurements reflect the code quality available to investors during their evaluation.

The repository states are then analyzed using the Qlty CLI platform. This tool was selected over other static analysis tools, such as SonarQube, due to its documented performance on large-scale codebases and its suitability for analyzing the varied projects in the sample. The pipeline manages Qlty execution across multiple repositories while handling memory constraints and timeout issues. The complete technical implementation is detailed in Appendix A.1.

### 3.2.2 Static Analysis Tool Selection and Validation

The selection of appropriate static analysis tools represents a critical methodological decision that directly impacts the validity and generalizability of technical debt measurements. This subsection details the systematic evaluation process used to select Qlty CLI as the primary analysis platform and addresses the implications of tool-specific measurement characteristics.

Table 3.2: Comparative Analysis of Static Analysis Platforms for Cross-Company Research

| Platform                            | Primary Focus                                                                          | TDR Approach                                                                     | Research Suitability                                                                                               |
|-------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| <b>SonarQube</b>                    | Continuous inspection of code quality, security, and reliability for development teams | Time-based remediation cost using default development rates (e.g., 30 min/KLOC)  | <b>High:</b> Open-source availability, extensive language support, standardized metrics, strong API access         |
| <b>CAST</b>                         | Portfolio-level analysis of software health and structural risk for enterprise systems | Business impact-focused debt calculation prioritizing high-risk structural flaws | <b>Medium:</b> Comprehensive analysis but proprietary algorithms, limited academic access                          |
| <b>Qlty (CodeClimate successor)</b> | Multi-language static analysis with configurable rule sets for large-scale codebases   | Effort-based remediation cost calculation with detailed issue categorization     | <b>High:</b> Scalable CLI interface, consistent cross-language analysis, suitable for automated research pipelines |

Qlty CLI was selected based on three primary criteria relevant to large-scale cross-company analysis: scalability for automated processing of diverse codebases, consistency of output for-

matting across different programming languages, and transparency of remediation cost calculations that enable reproducible TDR computation. The platform’s command-line interface proved essential for the automated pipeline developed for this research, enabling batch processing of repository states synchronized with funding timeline data.

However, the selection of any single static analysis tool introduces systematic measurement limitations that must be acknowledged. Empirical studies document significant divergence between tools when analyzing identical codebases, with overlap in identified issues typically less than 15% (Wang et al., 2018). Additionally, precision rates vary considerably, ranging from 20% to 80% depending on configuration (Beller et al., 2016). These inter-tool variations mean that TDR measurements become tool-specific rather than universally generalizable, establishing an important boundary condition for the interpretation of empirical relationships observed in this study.

The tool-specific measurement characteristics have direct implications for the strategic framework developed in this research. Companies classified as “high debt” based on Qlty analysis may receive different classifications from alternative static analysis platforms, suggesting that the strategic quadrants identified represent patterns specific to code-level issues detected by Qlty’s rule sets rather than comprehensive assessments of technical health across all debt dimensions.

### **3.2.3 Technical Debt Ratio Implementation and Economic Modeling**

Technical debt is quantified using the TDR framework, which provides an economic interpretation of code quality relevant to the financial context of venture capital evaluation. As defined in the literature review, TDR represents the ratio of remediation cost to total estimated development cost, enabling direct comparison of quality trade-offs across companies with different codebase sizes and development investments.

The remediation cost component is derived from the ‘totalEffortMinutes’ metric produced by Qlty analysis. This metric estimates the developer time required to address identified structural issues, coding standard violations, and complexity problems. The effort estimation is not based on traditional complexity metrics like Cyclomatic Complexity, but is instead grounded in the principles of Cognitive Complexity (Campbell, 2023). Cognitive Complexity is a modern metric designed to measure the understandability of code from a human perspective, penalizing structures that break the linear flow of code and require greater mental effort to comprehend, such as nested control flows and complex logical expressions. Qlty translates these principles into a quantifiable remediation cost by assigning a baseline effort in minutes to each type of issue (‘BASE\_EFFORT\_MINUTES’) and adding an incremental penalty based on the severity of the issue (‘EFFORT\_MINUTES\_PER\_VALUE\_DELTA’). This approach reflects the platform’s assessment of the “interest payments” associated with current technical debt levels—the additional development time required for future modifications due to existing quality

constraints.

The development cost denominator employs the Basic Constructive Cost Model (COCOMO) model’s organic mode for standardized cost estimation across the dataset. COCOMO estimates development effort as  $E = a \times (KLOC)^b$ , where the organic mode parameters ( $a = 2.4$ ,  $b = 1.05$ ) are specifically designed for small-to-medium software projects with stable requirements and experienced teams—characteristics that closely match the venture-backed startup environment analyzed in this study. The organic mode’s lower complexity exponent (1.05) reflects the relatively straightforward development challenges typical of early-stage software companies, making it the most appropriate COCOMO variant for this research context.

Crucially, while COCOMO’s absolute cost predictions exhibit documented variability in individual projects (Jeklin et al., 2025), the model’s strength lies in providing consistent relative estimates across organizations—precisely what is required for TDR calculation. The systematic application of identical parameters across all 70 companies ensures that observed differences in technical debt ratios reflect genuine variations in code quality rather than estimation inconsistencies. This methodological consistency enables the cross-company comparative analysis that forms the foundation of this research.

The TDR implementation acknowledges that the ratio primarily captures code-level debt detected by static analysis, potentially excluding architectural, test, and documentation debt that may influence startup velocity. However, code-level debt represents the most objectively measurable and comparable form of technical debt across organizations, providing a standardized foundation for empirical analysis. The economic framing through COCOMO enables direct comparison of quality investments across companies with different scales and development histories.

For the large-scale empirical analysis undertaken in this research, TDR provides the optimal balance of objective measurement, cross-company comparability, and theoretical grounding necessary to examine technical debt relationships in venture-backed environments. The consistent application of organic mode COCOMO parameters ensures that strategic insights emerge from genuine patterns in the data rather than methodological artifacts.

### **3.2.4 Theoretical Justification for TDR as Strategic Proxy**

While TDR calculations capture code-level manifestations of quality trade-offs, interpreting these patterns as evidence of strategic choices requires establishing theoretical connections between observable technical metrics and organizational decision-making processes. This theoretical foundation justifies using TDR patterns as proxies for startup strategic priorities rather than treating them as merely technical artifacts.

The theoretical bridge from code metrics to strategic intent rests on three complementary frameworks. First, resource allocation theory from organizational behavior (Cyert & March, 1963) explains that consistent patterns of code-level debt across multiple modules and development

periods reflect systematic organizational choices rather than random technical accidents. When teams repeatedly choose expedient implementations over architectural elegance across diverse technical contexts, these patterns reveal underlying resource allocation priorities embedded in organizational culture and incentive structures.

Second, applying behavioral signal theory (Spence, 1973) to the technical debt context, code-level debt accumulation patterns serve as observable indicators of organizational risk tolerance and temporal preferences. Teams that consistently accept higher complexity and defer quality improvements across multiple development cycles reveal their strategic orientation toward short-term achievement over long-term sustainability. Unlike traditional signaling contexts where agents actively manage their signals, technical debt patterns represent authentic behavioral indicators that emerge from genuine organizational priorities.

Third, constraint-based decision making literature (Simon, 1955) provides the economic rationality for systematic debt accumulation as optimal strategic choice. In venture-backed environments characterized by severe time constraints and competitive pressures, technical debt represents “temporal arbitrage”—systematically trading future development capacity for immediate execution capability when current capacity is more valuable than equivalent future capacity (Fisher, 1930).

This theoretical convergence establishes TDR as a valid proxy for organizational strategic intent regarding quality-speed trade-offs, enabling interpretation of observed empirical relationships as evidence of strategic rather than merely technical phenomena.

### **3.2.5 Composite Velocity Framework**

Development velocity is measured using the composite metric defined in Equation 2.1, adapted from the CNCF’s project velocity framework for the startup evaluation context.

The implementation extracts the required components from Git repository data for each development period. Code churn per author is calculated by summing lines added and deleted, normalized by period length and team size. Commits per author represents the development rhythm over the same normalized basis. Both metrics are then scaled by their respective dataset maximums to ensure the composite velocity ranges from 0 to 1, enabling meaningful cross-company comparison despite significant variations in team sizes across venture-backed startups.

The per-author normalization accounts for teams within the sample, while the max-normalization ensures that companies with different absolute activity levels can be meaningfully compared on execution intensity.

## **3.3 Statistical Challenges and Solutions**

A fundamental challenge in this longitudinal study is the nested structure of the data and ensuring appropriate statistical inference. This section addresses the key statistical issues and

presents the solutions employed to maintain analytical rigor.

### 3.3.1 Addressing the Nested Data Structure Problem

The dataset contains 146 development periods from 70 companies, meaning that some companies contribute multiple observations. This structure violates the independence assumption underlying standard correlation and regression analyses, as observations from the same company are likely to be correlated due to consistent organizational practices, team capabilities, and strategic approaches (Raudenbush & Bryk, 2002).

To address this statistical validity concern, this study employs a dual analytical approach that provides both granular and aggregate perspectives on the relationships of interest:

**Per-Period Analysis:** The primary descriptive analysis examines correlations across all 146 individual development periods, providing detailed insight into period-to-period variations in technical debt and velocity patterns. While this approach maximizes statistical power and captures the full range of strategic transitions within companies, it acknowledges that the independence assumption may be violated.

**Per-Company Analysis:** To ensure statistical validity, a complementary analysis aggregates all metrics to the company level by calculating mean values across all periods for each company. This approach transforms the nested structure into 70 independent company-level observations, satisfying the independence assumption required for correlation analysis. The per-company analysis serves as the primary basis for statistical inference, while the per-period analysis provides additional descriptive insight.

The implementation calculates company-level averages as:

For each company  $c$  with periods  $p_1, p_2, \dots, p_n$  :

$$\overline{\text{TDR}}_c = \frac{1}{n} \sum_{i=1}^n \text{TDR}_{p_i} \quad (3.1)$$

$$\overline{\text{Velocity}}_c = \frac{1}{n} \sum_{i=1}^n \text{Velocity}_{p_i} \quad (3.2)$$

$$\overline{\text{FundingGrowth}}_c = \frac{1}{n} \sum_{i=1}^n \text{FundingGrowth}_{p_i} \quad (3.3)$$

This dual approach allows for robust statistical inference while preserving the rich longitudinal information captured in the dataset.

### 3.3.2 Statistical Power and Inference Procedures

For both the per-period and per-company analyses, statistical significance is assessed using Pearson correlation coefficients with associated p-values calculated using the t-distribution. The degrees of freedom are adjusted appropriately:  $n - 2$  for the per-period analysis (144 degrees of

freedom) and  $c - 2$  for the per-company analysis (68 degrees of freedom), where  $c$  represents the number of companies.

Given the exploratory nature of this research and the relatively novel application domain, significance levels are interpreted using conventional thresholds ( $p < 0.05$  for significance,  $p < 0.10$  for marginal significance) while emphasizing effect sizes and practical significance alongside statistical significance.

The regression analysis provides  $R^2$  values to quantify the proportion of variance in development velocity explained by technical debt levels, offering insight into both the statistical and practical importance of the relationship.

### 3.4 Strategic Framework and Analysis Approaches

Beyond examining simple correlations, this study develops a strategic framework for understanding technical debt and velocity combinations. This section details the multi-framework approach used to test the robustness of strategic insights across different analytical perspectives.

#### 3.4.1 Primary Strategic Framework: The Debt-Velocity Matrix

The core analytical framework classifies companies into four strategic quadrants based on their technical debt and development velocity profiles. This matrix provides a practical tool for understanding the strategic implications of different debt-velocity combinations in venture-backed environments.

|               | Low Debt                                                                             | High Debt                                                                       |
|---------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| High Velocity | <b>Sustainable Growth</b><br>Low Debt + High Velocity<br>Traditional "best practice" | <b>Strategic Debt</b><br>High Debt + High Velocity<br>"Move fast, break things" |
| Low Velocity  | <b>Premature Optimization</b><br>Low Debt + Low Velocity<br>Over-engineering         | <b>Technical Burden</b><br>High Debt + Low Velocity<br>Worst of both worlds     |

Figure 3.2 Strategic Framework: Technical Debt-Velocity Matrix

The four strategic quadrants are defined as:

**Sustainable Growth (Low Debt + High Velocity):** Companies that achieve high development velocity while maintaining low technical debt levels. This represents the traditional engineering ideal of sustainable, high-quality development practices.

**Strategic Debt (High Debt + High Velocity):** Companies that accept high technical debt levels in pursuit of maximum development velocity. This quadrant embodies the "move fast and break things" philosophy and represents the key strategic question of this research.

**Premature Optimization (Low Debt + Low Velocity):** Companies with low technical debt but also low development velocity, potentially indicating over-engineering or analysis paralysis that constrains execution speed.

**Technical Burden (High Debt + Low Velocity):** Companies suffering from both high technical debt and low velocity, representing the worst-case scenario where debt accumulation has begun to significantly constrain development capability.

### 3.4.2 Comprehensive Sensitivity Analysis Framework

To ensure the robustness of findings across different statistical methodologies, this study implements a comprehensive sensitivity analysis that tests the strategic framework using multiple classification approaches. This addresses potential concerns that results might be artifacts of the specific median-split methodology initially employed.

**Primary Framework (Median Split):** Companies are classified into four strategic quadrants based on median splits of technical debt ratio and development velocity. This approach ensures balanced group sizes and provides clear strategic interpretations.

**Tertile Framework (3×3 Matrix):** To test for non-linear relationships and provide more granular strategic categories, the analysis implements a nine-quadrant framework using tertile splits. This approach divides each metric into Low (bottom 33%), Medium (middle 33%), and High (top 33%) categories, creating nine distinct strategic positions:

- Low Debt × {Low, Medium, High} Velocity
- Medium Debt × {Low, Medium, High} Velocity
- High Debt × {Low, Medium, High} Velocity

The tertile framework is particularly valuable for identifying whether performance relationships are monotonic or whether there are optimal “sweet spots” that emerge in the middle ranges of either metric.

**Quartile Extremes Framework (2×2 Focused):** To examine the most pronounced strategic differences, this analysis compares only the extreme quartiles (bottom 25% vs. top 25%) for both metrics, effectively filtering out the middle 50% of observations. This approach tests whether the relationships observed in the median split framework are driven primarily by the most extreme strategic positions or represent continuous relationships across the full range of observations.

The sensitivity analysis framework is designed to test three critical methodological assumptions:

1. **Threshold Sensitivity:** Whether findings are robust to different cut-point definitions
2. **Linearity Assumptions:** Whether relationships hold across the full distribution or only at extremes
3. **Strategic Distinctiveness:** Whether intermediate positions show meaningfully different patterns from extreme positions

## 3.5 Validation and Quality Assurance

This section consolidates the various approaches used to ensure the reliability and validity of the findings, including cross-framework validation, outlier management, and comprehensive assessment of study limitations.

### 3.5.1 Cross-Framework Validation

The multiple analytical frameworks serve as internal validation mechanisms. Consistent patterns across median, tertile, and quartile methodologies provide evidence that findings are not artifacts of specific analytical choices. Conversely, framework-specific results are flagged and interpreted with appropriate caution.

This validation approach is particularly important given the exploratory nature of the research questions. By demonstrating consistency across multiple analytical perspectives, the study provides greater confidence in the strategic implications derived from the data.

### 3.5.2 Outlier Management and Sensitivity

To ensure that results are not driven by extreme observations, the analysis implements systematic outlier detection using the interquartile range (IQR) method. Observations falling beyond  $1.5 \times \text{IQR}$  from the first and third quartiles are identified and, where appropriate, excluded from funding growth calculations while being retained for other analyses.

The sensitivity analysis framework also serves as a natural robustness check against outlier influence, as consistent patterns across different statistical cuts suggest that findings are not dominated by a small number of extreme cases.

For transparency, all outlier exclusions are documented and their impact on key findings is explicitly tested and reported.

### 3.5.3 Study Limitations and Mitigation Strategies

While these methodological enhancements significantly strengthen the statistical validity of the analysis, several limitations remain:

**Sample Representativeness:** The use of open-source companies, while necessary for data access, may not be fully representative of the broader proprietary software startup ecosystem.



Open-source projects often benefit from community contributions, which could influence both velocity and code quality in ways not typical for closed-source ventures (Colazo, 2010).

**Statistical Power Trade-offs:** The per-company analysis, while statistically appropriate, reduces the effective sample size from 146 to 70 observations, potentially reducing power to detect smaller effects. This trade-off is accepted in favor of statistical validity.

**Averaging Effects:** Company-level averaging may obscure important within-company variation in strategic approaches over time. The per-period analysis preserves this information for descriptive purposes while acknowledging its statistical limitations.

**Multiple Comparison Concerns:** The use of multiple analytical frameworks increases the risk of Type I errors. Results are interpreted with awareness of this limitation, emphasizing consistency across frameworks rather than isolated significant findings.

**Confounding Variables:** The relationship between debt, velocity, and funding is likely moderated by numerous factors not fully controlled for in this analysis, such as the startup's specific industry, stage of maturity, competitive intensity, and the size and experience of the development team.

**Metric Limitations:** Both TDR and the composite velocity metric are proxies with inherent limitations. TDR may not capture all forms of debt, particularly architectural and strategic debt, while velocity metrics do not inherently measure the market value or strategic importance of the features being produced.

**Temporal Dependencies:** Even at the company level, funding outcomes may exhibit temporal correlation due to broader market conditions during the ZIRP era. While the study cannot fully control for these macro-economic factors, the consistent time period across all observations provides some mitigation.

**Temporal and Contextual Constraints:** The findings from this study are specific to the macro-economic conditions of the ZIRP era, and may not generalize to different capital environments or economic cycles.

These methodological enhancements transform the analysis from a purely exploratory correlation study into a robust, multi-perspective examination of technical debt and velocity relationships that can support more confident conclusions about their strategic implications in the venture capital context.

# Results

This chapter presents the empirical findings from the analysis of 70 venture-backed software companies across 146 development periods during the ZIRP era (2009-2022). The analysis reveals a context-dependent view of technical debt management that challenges conventional software engineering wisdom. The findings demonstrate that development velocity, rather than technical debt avoidance, serves as the primary predictor of funding success, with strategic debt accumulation potentially enhancing rather than constraining venture outcomes when paired with rapid execution.

## 4.1 Sample Characteristics and Descriptive Analysis

The final analytical dataset comprises 146 development periods from 70 venture-backed companies, representing a 95.4% retention rate from the initial 153 potential development periods identified through systematic filtering. This high retention rate reflects the robust quality assurance applied to ensure analytical validity while maintaining sufficient sample size for meaningful statistical inference.

The filtering process, summarized in Table 4.1, was designed to ensure that TDR calculations reflect substantial software systems rather than early prototypes, and that development periods contain sufficient activity to generate meaningful velocity measurements. Of the 153 initial records, only 7 observations (4.6%) were filtered due to insufficient data quality across multiple dimensions, with the primary exclusions arising from minimal development activity rather than fundamental data availability issues.

Table 4.1: Data Quality Assessment and Filtering Results

| Quality Criterion                                     | Records    | Retention Rate |
|-------------------------------------------------------|------------|----------------|
| Initial development periods                           | 153        | 100.0%         |
| Valid TDR calculations ( $0 \leq \text{TDR} \leq 1$ ) | 153        | 100.0%         |
| Sufficient codebase size ( $\text{LOC} > 5,000$ )     | 151        | 98.7%          |
| Minimum period length ( $> 90$ days)                  | 149        | 97.4%          |
| Minimum development activity ( $> 10$ commits)        | 146        | 95.4%          |
| <b>Final analytical sample</b>                        | <b>146</b> | <b>95.4%</b>   |

The descriptive characteristics of the final sample, presented in Table 4.2, reveal the central

tendencies of the key analytical variables. Development periods averaged 489 days, reflecting the diversity of funding cycles across different venture stages and market conditions. Technical debt ratios averaged 0.031, indicating that most companies in the sample maintained relatively modest levels of code-level debt. Development velocity averaged 0.089, with funding growth rates averaging 151.3% between successive rounds.

Table 4.2: Descriptive Statistics for Key Variables (N = 146 periods)

| Variable                         | Mean  |
|----------------------------------|-------|
| Development period length (days) | 489.0 |
| Technical Debt Ratio             | 0.031 |
| Development Velocity (composite) | 0.089 |
| Funding Growth Rate (%)          | 151.3 |

Critically, the funding success rate across the sample is 54.1%, meaning that slightly more than half of the development periods were followed by successful subsequent funding rounds. This balanced outcome distribution provides optimal conditions for analyzing the factors associated with venture capital success, avoiding the statistical challenges that would arise from heavily skewed success rates.

The sample comprises companies spanning six primary industry categories within the venture-backed software ecosystem, as detailed in Table 4.3. The distribution reflects the broader venture ecosystem during the ZIRP era, with infrastructure and developer-focused companies representing the largest segments. Notably, Database & Storage companies achieved the highest funding success rate (64.3%) while maintaining relatively high technical debt levels (0.083 average TDR) and high development velocity (0.161), exemplifying the Strategic Debt quadrant. Security companies also showed strong performance (66.7% success rate) with the highest average technical debt levels (0.191 TDR). Conversely, Business & Application Software companies showed the lowest success rate (30.0%) despite moderate debt levels. This variation across industries provides natural controls for examining whether debt-velocity relationships hold consistently across different market contexts and technical domains.

Table 4.3: Industry Distribution and Performance Overview (N = 70 companies)

| Industry Category               | Companies | Success Rate | Avg TDR      | Avg Velocity |
|---------------------------------|-----------|--------------|--------------|--------------|
| Developer Tools & DevOps        | 23        | 58.7%        | 0.043        | 0.113        |
| Database & Storage              | 18        | 64.3%        | 0.083        | 0.161        |
| Data Platform & Analytics       | 12        | 50.0%        | 0.049        | 0.127        |
| Business & Application Software | 8         | 30.0%        | 0.063        | 0.108        |
| AI/Machine Learning             | 4         | 42.9%        | 0.054        | 0.054        |
| Security                        | 5         | 66.7%        | 0.191        | 0.108        |
| <b>Total</b>                    | <b>70</b> | <b>54.1%</b> | <b>0.031</b> | <b>0.089</b> |

## 4.2 Strategic Relationships: Debt, Velocity, and Funding Success

The analysis examines how technical debt and development velocity associate with funding outcomes, revealing unexpected patterns that challenge conventional assumptions. Understanding these relationships requires first establishing whether technical debt constrains development velocity as traditional engineering frameworks predict.

### 4.2.1 The Debt-Velocity Relationship

Examining the relationship between technical debt and development velocity, the company-level analysis reveals no significant relationship, though period-level patterns suggest potential complexity in this relationship. Table 4.4 presents correlation results using both per-period and per-company approaches to address potential data dependency concerns. The per-period analysis shows a statistically significant positive correlation ( $r = 0.229$ ,  $p = 0.005$ ) between technical debt and development velocity, suggesting that higher debt levels associate with faster development rather than the anticipated velocity constraints. Critically, when analyzed at the company level ( $r = 0.056$ ,  $p = 0.667$ ), where data independence assumptions are satisfied, no significant relationship emerges.

Given the methodological requirement for independent observations, subsequent analyses and conclusions are based on the company-level results, while the period-level patterns provide supplementary descriptive context.

Table 4.4: Correlation Analysis: Technical Debt, Velocity, and Funding Outcomes

| Relationship                               | Correlation (r) | p-value | Significance               |
|--------------------------------------------|-----------------|---------|----------------------------|
| <i>Per-period analysis (N = 146):</i>      |                 |         |                            |
| TDR $\leftrightarrow$ Development Velocity | 0.229           | 0.005   | Significant ( $p < 0.01$ ) |
| TDR $\leftrightarrow$ Funding Growth       | 0.134           | 0.308   | Not Significant            |
| Velocity $\leftrightarrow$ Funding Growth  | -0.248          | 0.052   | Not Significant            |
| <i>Per-company analysis (N = 70):</i>      |                 |         |                            |
| TDR $\leftrightarrow$ Development Velocity | 0.056           | 0.667   | Not Significant            |

With an  $R^2$  of 0.052, technical debt explains only 5.2% of the variance in development velocity, indicating that while statistically detectable, the practical relationship is weak. This finding suggests that technical debt accumulation may not impose the systematic velocity constraints assumed by traditional software engineering frameworks, at least within the range of debt levels observed in this venture-backed sample.

Turning to the relationship between development velocity and funding success, the analysis reveals additional complexity that requires examination through multiple lenses. Development velocity shows a negative correlation with funding growth rates ( $r = -0.248$ ,  $p = 0.052$ ), though

this relationship is not statistically significant at conventional levels. This pattern potentially reflects different valuation dynamics, where high-velocity companies may achieve more frequent but smaller funding increments, while low-velocity companies must demonstrate exceptional market traction to justify continued investment.

However, when examining funding success as a binary outcome—the ability to secure subsequent rounds regardless of growth magnitude—development velocity emerges as a critical success factor. This distinction between funding probability and funding growth magnitude becomes central to understanding the strategic implications, as it suggests that velocity primarily influences investor confidence in continued viability rather than driving dramatic valuation increases.

The weak statistical relationship between technical debt and development velocity, combined with the complex relationship between velocity and funding outcomes, indicates that simple bivariate correlations may be insufficient for understanding strategic trade-offs in venture-backed environments. These findings motivate the strategic framework analysis that examines combined debt-velocity effects on funding success, potentially revealing interaction effects that are not apparent in individual correlation analyses.

### **4.3 Strategic Framework and Robustness Analysis**

To examine the combined effects of technical debt and development velocity on funding success, companies were classified into four strategic quadrants based on median splits of both variables. This framework reveals how different debt-velocity combinations associate with venture capital outcomes, providing insight into the strategic viability of debt accumulation when paired with rapid execution.

The strategic framework results, presented in Figure 4.1 and detailed in Table 4.5, challenge conventional engineering wisdom by demonstrating that the Strategic Debt quadrant (high technical debt combined with high development velocity) achieves the highest funding success rate at 60.6%. This finding suggests that strategic debt accumulation, when combined with high execution velocity, may actually enhance rather than constrain funding prospects in venture-backed environments.

The Sustainable Growth quadrant (low debt + high velocity), representing traditional engineering best practices, achieves a 57.5% success rate—lower than the Strategic Debt approach. This pattern suggests that pure debt avoidance may not be optimal when it constrains the rapid iteration and feature delivery that venture capital markets reward. The Premature Optimization quadrant shows the lowest success rate at 45.5%, providing empirical evidence that over-engineering at the expense of velocity may be counterproductive in venture-backed environments where market timing and demonstrated momentum are critical.

Despite low success rates, companies in the Technical Burden quadrant achieve the highest av-

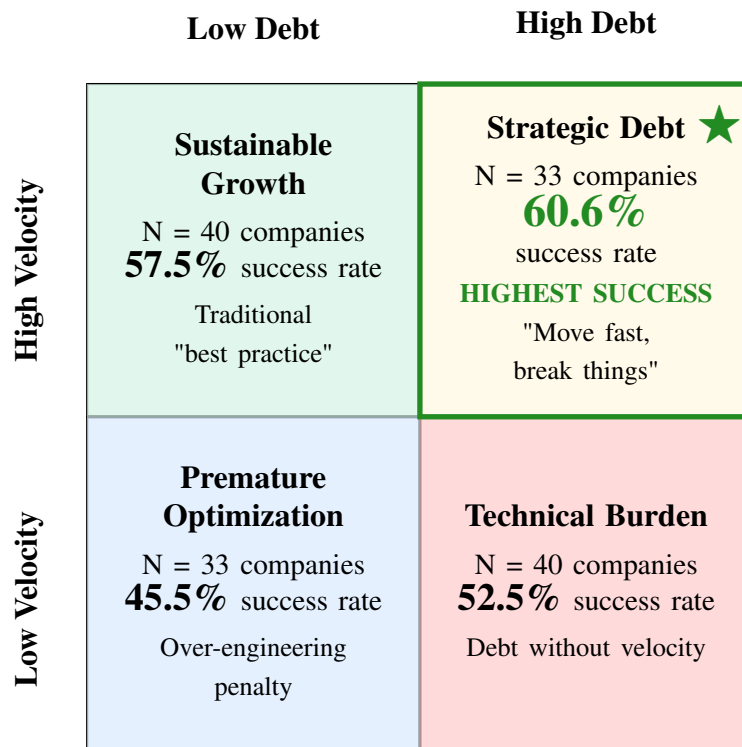


Figure 4.1 Strategic Framework Results: Funding Success by Debt-Velocity Combinations (N=146 periods)

Table 4.5: Strategic Framework: Performance Metrics by Quadrant

| Strategic Quadrant                                         | N  | Success Rate | Avg TDR | Avg Velocity | Funding Growth |
|------------------------------------------------------------|----|--------------|---------|--------------|----------------|
| <b>Sustainable Growth</b><br>(Low Debt + High Velocity)    | 40 | 57.5%        | 0.014   | 0.185        | 144.5%         |
| <b>Strategic Debt</b><br>(High Debt + High Velocity)       | 33 | 60.6%        | 0.133   | 0.229        | 161.7%         |
| <b>Premature Optimization</b><br>(Low Debt + Low Velocity) | 33 | 45.5%        | 0.013   | 0.048        | 141.4%         |
| <b>Technical Burden</b><br>(High Debt + Low Velocity)      | 40 | 52.5%        | 0.091   | 0.046        | 272.6%         |

verage funding growth (272.6%) when successful. This suggests that while investors are initially skeptical of their technical capabilities, exceptional market traction can overcome these doubts. The subsequent large funding rounds then serve as both a catch-up investment and a premium for the high risk undertaken

Examining the patterns across quadrants reveals a consistent velocity effect. Table 4.6 isolates this by comparing success rates across debt levels while controlling for velocity differences. Among low-debt companies, high velocity provides a 12.0 percentage point advantage in funding success, while among high-debt companies, high velocity provides an 8.1 percentage point advantage. This consistent "velocity premium" suggests that rapid execution capability represents a fundamental competitive advantage in venture-backed environments, with debt levels playing a secondary moderating role.

Table 4.6: Development Velocity Effect on Funding Success by Debt Level

| Debt Level          | Low Velocity | High Velocity | Velocity Premium |
|---------------------|--------------|---------------|------------------|
| Low Technical Debt  | 45.5%        | 57.5%         | +12.0 pp         |
| High Technical Debt | 52.5%        | 60.6%         | +8.1 pp          |

### 4.3.1 Robustness Testing Across Classification Schemes

To test whether these findings are artifacts of the median-split methodology, the analysis was repeated using tertile (3×3) and quartile extreme (2×2) classification schemes. The tertile analysis, dividing each metric into low, medium, and high categories, reinforces the velocity-driven pattern while revealing additional nuances in optimal debt levels. The detailed tertile results are provided in Appendix A.1, but the key finding remains consistent: high-velocity development associates with superior funding outcomes across different debt levels.

Building on the tertile analysis, the quartile extremes framework examines only the most pronounced strategic positions by comparing companies in the bottom 25% versus top 25% for both metrics. Table 4.7 demonstrates that the Very High Debt + Very High Velocity combination achieves the highest success rate (66.7%), while Very Low Debt + Very Low Velocity shows the lowest success rate (30.0%). This 36.7 percentage point difference between strategic extremes reinforces the primacy of velocity over debt avoidance in determining funding success.

Table 4.7: Quartile Extremes Framework: Performance at Strategic Extremes

| Extreme Position                    | N  | Success Rate | Avg Funding Growth |
|-------------------------------------|----|--------------|--------------------|
| Very Low Debt + Very Low Velocity   | 10 | 30.0%        | 73.0%              |
| Very High Debt + Very Low Velocity  | 12 | 50.0%        | 366.5%             |
| Very Low Debt + Very High Velocity  | 8  | 62.5%        | 102.8%             |
| Very High Debt + Very High Velocity | 12 | 66.7%        | 148.6%             |

The cross-framework validation, illustrated in Figure 4.2, demonstrates strong consistency across all three analytical approaches. Each framework confirms that high-velocity development associates with superior funding outcomes across different debt levels and classification schemes, providing confidence that the strategic insights represent genuine empirical patterns rather than methodological artifacts.

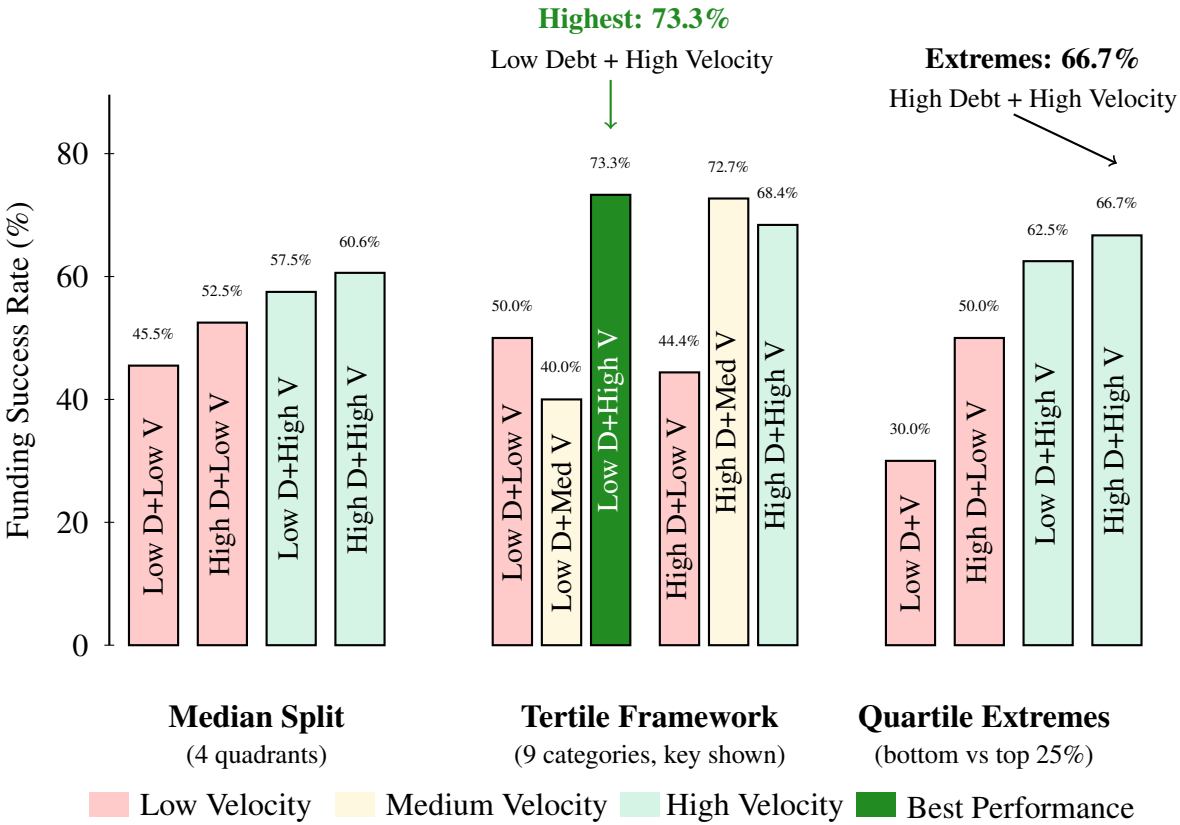


Figure 4.2 Cross-Framework Validation: Consistency of Velocity Effect Across Analysis Methods

### 4.3.2 Performance by Individual Metrics

To provide additional context for these strategic patterns, we examined funding success across technical debt and velocity quartiles independently. When examining technical debt quartiles (Table 4.8), the results challenge conventional assumptions by showing that companies in the lowest technical debt quartile achieve the lowest funding success rate (44.4%). Success rates peak in the second quartile (61.1%), suggesting that moderate technical debt levels may be optimal rather than minimal debt accumulation. The highest debt quartile maintains strong performance (57.9%) alongside the highest funding growth rates, indicating that investors do not systematically penalize technical debt when other performance indicators are favorable. In contrast, the analysis of development velocity quartiles (Table 4.9) reveals a clear monotonic relationship. Funding success rates increase consistently with development velocity, from 47.2% in the lowest quartile to 68.4% in the highest quartile, representing a 21.2 percentage



Table 4.8: Funding Performance by Technical Debt Ratio Quartiles

| <b>TDR Quartile</b> | <b>N</b> | <b>Avg TDR</b> | <b>Success Rate</b> | <b>Funding Growth</b> | <b>Avg Velocity</b> |
|---------------------|----------|----------------|---------------------|-----------------------|---------------------|
| Q1 (Lowest Debt)    | 36       | 0.006          | 44.4%               | 96.6%                 | 0.109               |
| Q2                  | 36       | 0.021          | 61.1%               | 194.0%                | 0.139               |
| Q3                  | 36       | 0.050          | 52.8%               | 185.1%                | 0.104               |
| Q4 (Highest Debt)   | 38       | 0.165          | 57.9%               | 238.4%                | 0.149               |

point advantage for high-velocity development. This monotonic relationship strongly supports the velocity-focused strategy implications emerging from the quadrant analysis.

Table 4.9: Funding Performance by Development Velocity Quartiles

| <b>Velocity Quartile</b> | <b>N</b> | <b>Avg Velocity</b> | <b>Success Rate</b> | <b>Funding Growth</b> | <b>Avg TDR</b> |
|--------------------------|----------|---------------------|---------------------|-----------------------|----------------|
| Q1 (Lowest Velocity)     | 36       | 0.029               | 47.2%               | 263.9%                | 0.064          |
| Q2                       | 36       | 0.063               | 50.0%               | 160.1%                | 0.047          |
| Q3                       | 36       | 0.117               | 50.0%               | 162.7%                | 0.045          |
| Q4 (Highest Velocity)    | 38       | 0.285               | 68.4%               | 170.4%                | 0.090          |

Cross-industry validation demonstrates that these strategic patterns hold consistently across different technical domains, as shown in the sample characteristics analysis. This consistency suggests that the identified relationships reflect fundamental venture capital evaluation dynamics rather than domain-specific phenomena.

## 4.4 Synthesis of Key Findings

The empirical analysis yields a coherent set of findings that fundamentally challenge conventional assumptions about technical debt management in startup environments. These results demonstrate that technical debt's strategic impact is context-dependent, with debt accumulation potentially serving as a tool for competitive advantage rather than an unqualified liability when paired with rapid execution capability.

First, technical debt does not systematically constrain development velocity within venture-backed environments ( $r = 0.056$ ,  $p = 0.667$ , company-level). This finding is crucial because it explains why strategic debt accumulation can succeed: debt doesn't automatically slow teams down within typical startup ranges. Technical debt explained only 5.2% of velocity variance ( $R^2 = 0.052$ ), indicating that other factors—team capability, architectural decisions, development infrastructure—dominate execution speed.

Second, the strategic framework analysis establishes that development velocity consistently emerges as the primary success driver across all debt levels. The Strategic Debt quadrant (high debt + high velocity) achieves the highest funding success rate at 60.6%, while the velocity premium of 8.1-12.0 percentage points remains consistent regardless of debt levels. This pattern holds robustly across multiple analytical frameworks, providing strong evidence for its strategic

importance.

The robustness analysis confirms these strategic insights while revealing additional nuances. Moderate technical debt levels may be optimal rather than minimal accumulation, and the velocity advantage remains consistent across different classification schemes and statistical approaches. The finding that companies with the lowest technical debt achieve the lowest funding success rate (44.4%) particularly challenges traditional engineering wisdom, while the monotonic relationship between velocity and success (47.2% to 68.4% across quartiles) reinforces the primacy of execution speed.

These findings provide empirical evidence for a context-dependent view of technical debt management that acknowledges the unique incentive structures and competitive pressures of venture-backed environments. During periods of capital abundance such as the ZIRP era, the market appears to reward demonstrated execution velocity over internal code quality metrics, supporting strategic frameworks where technical debt accumulation serves as "temporal arbitrage"—trading future development capacity for immediate competitive advantage when current execution capability is more valuable than equivalent future capacity. This represents a fundamental departure from traditional software engineering approaches that universally characterize technical debt as a liability to be minimized, suggesting instead that optimal strategies depend critically on macroeconomic conditions, competitive dynamics, and organizational objectives.

# Discussion

---

This study reveals that technical debt's strategic impact in venture-backed environments is context-dependent rather than universally negative. The empirical findings establish development velocity as the primary driver of funding success while demonstrating that technical debt can serve as a competitive tool under specific conditions. These counter-intuitive results demand careful examination of what they mean for both entrepreneurial practice and software engineering theory.

## 5.1 What the Debt-Velocity Disconnect Really Means

The absence of a significant relationship between technical debt and development velocity ( $r = 0.056$ ,  $p = 0.667$ ) represents more than a statistical null result—it reveals important boundary conditions for traditional technical debt theory. Within the range of debt levels observed in venture-backed companies, technical debt accumulation does not systematically constrain development velocity as conventional wisdom predicts.

This disconnect exposes critical limitations in established technical debt theory. Cunningham's debt metaphor assumes that "interest payments" inevitably accumulate and constrain future development capacity (Cunningham, 1992). However, this assumption appears to break down in venture-backed environments where external funding can continuously subsidize technical inefficiencies while market timing pressures make execution speed existentially important.

The practical implication merits consideration: entrepreneurs may be over-investing in code quality at the expense of market responsiveness. Within typical startup technical debt ranges (mean TDR = 0.031), quality trade-offs do not meaningfully impact execution capability. This creates space for what we term "temporal arbitrage"—a strategic approach where organizations systematically trade future development efficiency for immediate execution capability when current capacity is demonstrably more valuable than equivalent future capacity.

The theoretical foundation for temporal arbitrage draws from financial theory's concept of intertemporal choice (Fisher, 1930), where rational actors optimize across time periods based on relative value and discount rates. In venture-backed contexts, the "discount rate" for future technical capacity is effectively determined by competitive dynamics and capital availability. When external funding can subsidize future remediation costs and competitive pressures cre-

ate winner-take-all dynamics, the present value of immediate execution capability exceeds the discounted cost of future technical constraints.

However, technical debt explains only 5.2% of velocity variance ( $R^2 = 0.052$ ), indicating that other factors, like team capability, architectural decisions, tooling, and organizational processes, dominate development speed. This weak relationship implies that technical debt may be less strategically significant than either engineering literature or this study's analysis framework assumes. For resource-constrained startups, investments in team quality and development infrastructure may yield substantially higher velocity returns than investments in code-level quality maintenance.

## 5.2 The Strategic Debt Success Pattern

The Strategic Debt quadrant's superior performance (60.6% success rate) compared to traditional best practices (57.5% for Sustainable Growth) demonstrates how market dynamics can invert conventional engineering priorities. This pattern holds consistently across multiple analytical frameworks, indicating a genuine strategic phenomenon rather than statistical artifact.

Several mechanisms may explain why strategic debt accumulation enhances rather than constrains venture outcomes. First, the consistent velocity premium of 8.1-12.0 percentage points across debt levels demonstrates that rapid execution capability provides competitive advantages that outweigh code quality concerns. High-velocity teams can respond faster to market feedback, iterate more quickly through product hypotheses, and demonstrate momentum that attracts both customers and investors.

Second, information asymmetry theory (Spence, 1973) provides a framework for understanding venture capital evaluation dynamics. Investors typically lack the technical expertise necessary for comprehensive code quality assessment, making observable execution patterns critical screening mechanisms. The ability to consistently ship features and iterate rapidly provides tangible evidence of team effectiveness that transcends subjective technical evaluations. This creates a signaling environment where demonstrated velocity becomes a more reliable indicator of team capability than underlying code quality.

The poor performance of the Premature Optimization quadrant (45.5% success rate) provides evidence that over-engineering can be counterproductive in venture contexts. Companies in this quadrant maintain excellent code quality but fail to demonstrate the market responsiveness that venture markets reward during the ZIRP era. This pattern implies that perfectionist tendencies, while valuable in mature organizations, may actually harm early-stage venture prospects by constraining market learning and adaptation speed.

The Technical Burden quadrant's paradoxical high funding growth rates (272.6%) when successful illustrate how exceptional market traction can overcome technical skepticism. However, the low success probability (52.5%) indicates that this represents a high-risk strategy that suc-

ceeds only when product-market fit is exceptionally strong.

## **5.3 Why Velocity Dominates in Venture Context**

The monotonic relationship between development velocity and funding success—from 47.2% in the lowest quartile to 68.4% in the highest—establishes velocity as the dominant success factor across all technical debt levels. This relationship persists across industries, funding stages, and analytical frameworks, indicating a systematic dynamic rather than domain-specific phenomenon.

Venture capital markets during the ZIRP era explicitly rewarded demonstrated momentum over operational efficiency. With abundant capital available, the primary competitive differentiator became speed of market capture rather than technical excellence. The penalty for slow market execution exceeded the long-term costs of debt accumulation, creating incentives that directly opposed traditional engineering optimization criteria.

Building on dynamic capabilities theory (Teece et al., 1997), velocity generates positive feedback loops that extend beyond immediate technical execution. Teams demonstrating consistent delivery capability attract stronger talent, command greater investor confidence, and maintain higher organizational morale—each reinforcing capacity for continued rapid execution. This creates a dynamic capability that becomes embedded in organizational culture and processes, providing sustainable competitive advantages.

The temporal nature of venture competition amplifies velocity's importance. Market windows are often brief, first-mover advantages significant, and competitive responses rapid. Organizations capable of fast iteration can exploit opportunities that disappear before slower competitors can respond, making current execution capacity more valuable than equivalent future capacity achieved through quality investments.

## **5.4 Alternative Explanations and Competing Interpretations**

Several alternative explanations could account for the observed patterns, each with different implications for generalizability and practical application. Sample selection bias represents the most significant concern. Open-source companies may exhibit different technical debt dynamics than proprietary software companies due to community contributions that offset internal debt costs and transparency requirements that incentivize different quality-velocity trade-offs. Measurement limitations may obscure true relationships. The Technical Debt Ratio captures only code-level debt detectable through static analysis, potentially missing architectural debt and strategic technical decisions that most directly impact velocity. Companies classified as "high debt" based on coding standards violations may maintain excellent architectural decisions that enable rapid development, while companies with clean code-level metrics may harbor

architectural constraints that limit scaling velocity.

Reverse causality offers another interpretation. Rather than debt accumulation enabling velocity, high-velocity teams may simply have different tolerance thresholds for technical imperfection, leading to apparent debt accumulation without genuine velocity constraints. The correlation between debt and success may reflect organizational risk appetite and capability rather than strategic debt management. Industry-specific effects could also drive the observed patterns. Infrastructure and developer tools companies, which dominate the sample, may be more tolerant of technical debt than consumer-facing applications where user experience depends more directly on code quality. The high success rates observed in Database & Storage (64.3%) and Security companies (66.7%) despite high debt levels may not generalize to other domains. The ZIRP era's unique macroeconomic conditions may have created temporary incentive distortions that do not represent sustainable strategic principles. The relationships observed under capital abundance may reverse under capital scarcity, where operational efficiency becomes more important than growth velocity.

## **5.5 Implications for Theory and Practice**

### **5.5.1 Theoretical Contributions**

This study extends three foundational frameworks in software engineering and entrepreneurship research, demonstrating how macroeconomic context fundamentally alters the strategic calculus of technical debt management.

First, the findings directly challenge Cunningham's debt metaphor (Cunningham, 1992), which assumes that "interest payments" inevitably accumulate and constrain development capacity. The absence of significant debt-velocity correlation ( $r = 0.056$ ,  $p = 0.667$ ) at the company level reveals a critical boundary condition: within venture-backed environments characterized by external capital abundance, the metaphor's core assumption breaks down. Cunningham's framework implicitly assumes organizations must pay technical debt costs from their own operational capacity. However, when external funding can continuously subsidize these costs—as occurred during the ZIRP era—the metaphor requires extension. Technical debt becomes less analogous to financial debt requiring repayment from revenues and more similar to subsidized borrowing where external parties (venture capitalists) willingly absorb the interest payments in exchange for market position and growth potential.

Second, this research provides empirical validation and extension of Fowler's Technical Debt Quadrant framework (Fowler, 2009). Fowler distinguished between deliberate-prudent debt (conscious strategic decisions) and reckless-inadvertent debt (poor practices without strategic intent), arguing that only the former could be justified as rational choice. The Strategic Debt quadrant identified in this study—achieving the highest funding success rate at 60.6%—represents empirical evidence that deliberate debt accumulation can indeed serve as competitive

advantage under specific conditions. However, the findings extend Fowler's framework in a crucial dimension: Fowler's model treats the strategic viability of debt as primarily an internal organizational decision about technical sophistication and planning capability. This study demonstrates that external macroeconomic conditions—capital abundance, investor priorities, competitive dynamics—fundamentally determine whether deliberate debt accumulation succeeds or fails. The same debt accumulation strategy that proves optimal during the ZIRP era may become catastrophic under capital scarcity, suggesting that Fowler's quadrant framework requires a third dimension representing external environmental conditions.

Third, the research contributes to dynamic capabilities theory (Teece et al., 1997) by identifying development velocity as a meta-capability that generates compound advantages in venture contexts. The monotonic relationship between velocity and funding success (47.2% to 68.4% across quartiles) demonstrates that execution speed functions as a foundational dynamic capability—enabling faster sensing through rapid experimentation, more effective seizing through quick market response, and superior transformation through shortened iteration cycles. However, the Strategic Debt finding extends this theory by revealing that dynamic capabilities can be deliberately accelerated through strategic technical trade-offs. Organizations accepting higher technical debt in exchange for velocity gains are effectively using temporal arbitrage to amplify their dynamic capabilities during critical competitive windows.

These theoretical contributions converge on a central insight: optimal technical strategies cannot be derived from universal engineering principles but must be understood as contextually embedded in macroeconomic conditions, competitive dynamics, and organizational resource constraints. The temporal arbitrage framework introduced here—where organizations strategically trade future technical efficiency for immediate execution capability when current capacity is demonstrably more valuable—provides one such contingent model requiring further theoretical development to establish clear boundary conditions.

## **5.6 Implications for Theory and Practice**

### **5.6.1 Extending Technical Debt Theory**

This study extends two foundational frameworks in software engineering by demonstrating that technical debt's impact depends critically on macroeconomic context.

First, the findings challenge Cunningham's debt metaphor (Cunningham, 1992), which assumes that "interest payments" inevitably constrain development capacity. The absence of significant debt-velocity correlation ( $r = 0.056$ ,  $p = 0.667$ ) reveals a critical boundary condition: when external funding can continuously subsidize technical inefficiencies—as during the ZIRP era—the metaphor's core assumption breaks down. Technical debt is not quite the same as financial debt, which you pay back using money you earn. It is more like a type of borrowing where other parties take on the interest payments in exchange for a share of the market.

Second, this research provides empirical validation of Fowler’s Technical Debt Quadrant framework (Fowler, 2009) while extending it in a crucial dimension. Fowler distinguished between deliberate-prudent debt and reckless-inadvertent debt, arguing that only conscious strategic decisions could be justified. The Strategic Debt quadrant’s superior performance (60.6% success rate) confirms that deliberate debt accumulation can serve as competitive advantage. However, Fowler’s model treats debt viability as primarily an internal organizational decision. This study demonstrates that external macroeconomic conditions—capital abundance, investor priorities, competitive dynamics—fundamentally determine whether deliberate debt accumulation succeeds or fails, suggesting Fowler’s framework requires a third dimension representing environmental context.

These contributions converge on a central insight: optimal technical strategies cannot be derived from universal engineering principles but must be understood as contextually embedded in macroeconomic conditions and competitive dynamics. The temporal arbitrage framework introduced here provides one such contingent model, though further theoretical development is necessary to establish clear boundary conditions.

### **5.6.2 Practical Implications**

For entrepreneurs, the Strategic Debt quadrant’s superior performance (60.6% vs. 57.5% for traditional best practices) indicates that perfectionist tendencies regarding code quality may actively harm venture prospects by constraining market learning velocity. Operationally, this implies redirecting engineering investments from code-level quality maintenance toward velocity enablers: deployment automation, rapid experimentation infrastructure, and team capability development. The decision framework provides specific guidance: prioritize velocity over code quality when market timing creates first-mover advantages, competitive dynamics reward rapid delivery, external funding can subsidize future remediation costs, and customer feedback loops require rapid iteration. However, this strategy demands organizational capabilities to avoid the Technical Burden scenario (52.5% success rate) where debt accumulation occurs without corresponding velocity gains.

For engineering leaders, the weak debt-velocity relationship ( $R^2 = 0.052$ ) indicates that typical startup debt levels do not meaningfully constrain execution speed. Investments in tooling, automation, and team capability yield higher velocity returns than code-level quality maintenance. The tertile analysis revealing that moderate debt levels may be optimal (61.1% success rate in Q2 vs. 44.4% in Q1) suggests that zero-debt targets may actually harm venture prospects.

For venture capital investors, development velocity emerges as a stronger due diligence signal than traditional technical quality assessments during capital-abundant periods. The monotonic velocity-success relationship (47.2% to 68.4% across quartiles) indicates that teams demonstrating rapid execution deserve higher confidence than teams with excellent code quality but slower delivery. The finding that companies with lowest technical debt achieve lowest fund-



ing success (44.4%) challenges conventional assumptions that code quality should dominate technical evaluation. Instead, investors should evaluate whether technical debt patterns reflect strategic choices that enable velocity, or organisational dysfunction that constrains execution.

## 5.7 Study Boundaries and Future Research Directions

This study’s findings are bounded by specific methodological choices and environmental conditions that establish both the scope of valid inference and productive directions for investigating the unique debt-velocity relationship identified here.

The most fundamental constraint concerns temporal specificity. This study examines venture-backed companies exclusively during the ZIRP era (2009-2022), when near-zero interest rates created unprecedented capital abundance. The strategic relationships identified may represent temporary dynamics specific to this macroeconomic environment rather than durable principles. I deliberately focused on this period to isolate relationships under extreme capital abundance, but the post-ZIRP transition beginning in 2022 provides a critical natural experiment: if velocity-focused strategies prove less effective under capital scarcity, it would validate the context-dependent framework while limiting practical applicability to future cohorts.

The sample composition introduces systematic bias. The reliance on open-source companies, necessary for data access, means these findings may not generalize to proprietary software startups. Open-source projects benefit from community contributions, face different transparency incentives, and concentrate in infrastructure rather than consumer applications. The sample’s concentration in infrastructure and developer tools (41 of 70 companies) limits generalizability to domains where user experience depends more directly on code quality. Additionally, the TDR measurement captures only code-level debt detectable through static analysis, potentially missing architectural and test debt that may have stronger velocity impacts.

The observational design means that strong causal claims cannot be made. I did not control for founder experience, network effects, competitive intensity, or team composition—all of which may confound observed relationships between technical debt, velocity, and funding success. Reverse causality remains possible: high-velocity teams may simply tolerate technical imperfection differently rather than debt accumulation enabling velocity.

Finally, measuring funding success as the ability to secure subsequent rounds captures only short-term viability rather than long-term sustainability or exit outcomes. Strategic Debt companies may achieve multiple funding rounds while creating constraints that eventually limit growth or reduce valuations. The unique relationship this study identifies, that strategic debt accumulation can enhance venture outcomes under specific macroeconomic conditions, requires longitudinal investigation to determine whether it represents sustainable strategy or temporary arbitrage that eventually imposes delayed costs.

# Conclusion

---

This study empirically examined the relationship between technical debt accumulation, development velocity, and funding success in venture-backed software companies during the ZIRP era (2009-2022). Through analysis of 70 companies across 146 funding periods, the research provides evidence that challenges conventional assumptions about technical debt management in entrepreneurial contexts.

The research reveals that technical debt's impact on venture success is fundamentally context-dependent rather than universally negative. The analysis established two critical findings. First, technical debt does not systematically constrain development velocity at the company level ( $r = 0.056$ ,  $p = 0.667$ ), contradicting the widely assumed negative correlation in software engineering literature. Technical debt explained only 5.2% of velocity variance ( $R^2 = 0.052$ ), indicating that other factors—team capability, architectural decisions, development infrastructure—dominate execution speed within typical startup debt ranges.

Second, development velocity emerged as the dominant predictor of funding success, with high-velocity development consistently associated with superior outcomes across all debt levels. The strategic framework analysis revealed that companies combining high technical debt with high velocity achieved the highest funding success rate (60.6%), outperforming traditional engineering best practices of low debt with high velocity (57.5%). This pattern held robustly across multiple analytical frameworks and industry categories.

These findings carry significant strategic implications. Under the capital-abundant conditions of the ZIRP era, strategic debt accumulation paired with rapid execution capability enhanced rather than constrained funding prospects. The market rewarded demonstrated momentum over internal code quality, creating incentives that directly opposed traditional software engineering optimization criteria. This suggests that entrepreneurs may systematically over-invest in code quality at the expense of market responsiveness during early venture stages when external funding can subsidize technical inefficiencies while competitive pressures make execution speed existentially important.

This research makes three core contributions to the intersection of software engineering and entrepreneurship. First, it provides systematic empirical evidence that debt-velocity relationships are contingent on macroeconomic and competitive contexts rather than following universal engineering principles. The absence of significant debt-velocity correlation within venture-backed environments establishes important boundary conditions for traditional technical debt

theory. Second, the automated analysis pipeline developed for this study enables scalable, cross-company examination of technical debt patterns, addressing methodological limitations in prior research that relied predominantly on single-company case studies. The open-source implementation (Appendix A.1) provides a foundation for extending this research to different economic periods and company populations. Third, the strategic framework introduces theoretical foundations for understanding when debt accumulation represents rational temporal arbitrage—trading future development efficiency for immediate execution capability when current capacity is demonstrably more valuable than equivalent future capacity.

However, these findings are bounded by significant contextual constraints that limit their generalizability. The analysis is specific to the unique macroeconomic conditions of the ZIRP era, when abundant capital fundamentally altered the strategic calculus surrounding quality-speed trade-offs. The reliance on open-source companies, while necessary for data access, introduces potential sample bias, as these organizations may exhibit different technical debt dynamics due to community contributions and transparency requirements. Additionally, the technical debt measurements capture primarily code-level issues detectable through static analysis, potentially missing architectural and strategic debt forms that may have stronger impacts on development velocity and long-term sustainability. The post-ZIRP transition beginning in 2022 provides a critical natural experiment for testing the durability of these relationships. If velocity-focused strategies prove less effective under capital scarcity and heightened profitability requirements, it would strongly validate the theoretical framework of context-dependent engineering optimization. Future research should examine whether the observed patterns persist across different economic cycles, extend the analysis to proprietary software companies, and investigate the organizational capabilities that enable successful strategic debt management without creating technical burden scenarios where debt accumulation constrains rather than enables execution. The complete methodology and implementation (Appendix A.1) enable researchers to reproduce this analysis or apply it to different datasets and economic periods.

This study establishes that optimal technical strategies must incorporate external economic and competitive factors rather than relying on universal engineering principles. The evidence suggests that software engineering theory requires more sophisticated, contingent frameworks that explicitly acknowledge how macroeconomic conditions, competitive dynamics, and organizational objectives influence the strategic value of technical debt accumulation. While the ZIRP era may represent a unique historical period, the underlying principle—that engineering decisions should be evaluated within their strategic and economic context—has broader implications for understanding entrepreneurial success. The entrepreneurs who thrived during this period were those who recognized that market timing and execution velocity could outweigh code perfection, embracing a fundamentally different calculus than traditional software development. Perhaps the most successful founders have always intuitively understood what this research now documents: that in the race to build the future, sometimes the wisest choice is not to build it perfectly, but to build it first.

---

# List of Figures

---

|     |                                                                                                                  |    |
|-----|------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Research Gap at the Intersection of Technical Debt, Startup Strategy, VC Funding, and ZIRP-Era Context . . . . . | 2  |
| 1.2 | Conceptual Framework - Development Velocity as Primary Success Driver . . .                                      | 4  |
| 2.1 | Fowler's Technical Debt Quadrant . . . . .                                                                       | 7  |
| 2.2 | Timeline of the Zero Interest-Rate Policy Era (2009-2022) . . . . .                                              | 14 |
| 3.1 | Research Methodology Overview . . . . .                                                                          | 17 |
| 3.2 | Strategic Framework: Technical Debt-Velocity Matrix . . . . .                                                    | 24 |
| 4.1 | Strategic Framework Results: Funding Success by Debt-Velocity Combinations (N=146 periods) . . . . .             | 32 |
| 4.2 | Cross-Framework Validation: Consistency of Velocity Effect Across Analysis Methods . . . . .                     | 34 |
| A.1 | Automated Analysis Pipeline Architecture . . . . .                                                               | 52 |

---

# List of Tables

---

|     |                                                                                                    |    |
|-----|----------------------------------------------------------------------------------------------------|----|
| 2.1 | Velocity Measurement Approaches for Cross-Company Startup Analysis . . . .                         | 12 |
| 3.1 | Distribution of Companies by Industry Category . . . . .                                           | 18 |
| 3.2 | Comparative Analysis of Static Analysis Platforms for Cross-Company Research                       | 19 |
| 4.1 | Data Quality Assessment and Filtering Results . . . . .                                            | 28 |
| 4.2 | Descriptive Statistics for Key Variables (N = 146 periods) . . . . .                               | 29 |
| 4.3 | Industry Distribution and Performance Overview (N = 70 companies) . . . . .                        | 29 |
| 4.4 | Correlation Analysis: Technical Debt, Velocity, and Funding Outcomes . . . .                       | 30 |
| 4.5 | Strategic Framework: Performance Metrics by Quadrant . . . . .                                     | 32 |
| 4.6 | Development Velocity Effect on Funding Success by Debt Level . . . . .                             | 33 |
| 4.7 | Quartile Extremes Framework: Performance at Strategic Extremes . . . . .                           | 33 |
| 4.8 | Funding Performance by Technical Debt Ratio Quartiles . . . . .                                    | 35 |
| 4.9 | Funding Performance by Development Velocity Quartiles . . . . .                                    | 35 |
| A.1 | Tertile Framework: Success Rates by Detailed Strategic Position . . . . .                          | 53 |
| A.2 | Venture-Backed Open-Source Software Companies: Funding Timeline Dataset<br>(Part 1 of 3) . . . . . | 54 |
| A.3 | Venture-Backed Open-Source Software Companies: Funding Timeline Dataset<br>(Part 2 of 3) . . . . . | 55 |
| A.4 | Venture-Backed Open-Source Software Companies: Funding Timeline Dataset<br>(Part 3 of 3) . . . . . | 56 |

---

# References

---

- Agarwal, R., & Gort, M. (2001, April). First-mover advantage and the speed of competitive entry, 1887–1986. *The Journal of Law and Economics*, 44(1), 161–177. doi: 10.1086/320279
- Arcelli Fontana, F., Ferme, V., & Spinelli, S. (2012). Investigating the impact of code smells debt on quality code evaluation. In *Proceedings of the 3rd international workshop on managing technical debt (mtd)* (pp. 15–22). IEEE. doi: 10.1109/MTD.2012.6225996
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... Thomas, D. (2001, February). *Manifesto for agile software development*. <http://agilemanifesto.org/>.
- Beller, M., Bholanath, R., McIntosh, S., & Zaidman, A. (2016). Analyzing the state of static analysis: A large-scale evaluation in open source software. In *2016 IEEE 23rd international conference on software analysis, evolution, and reengineering (saner)* (Vol. 1, pp. 470–481). doi: 10.1109/SANER.2016.105
- Besker, T., Martini, A., Lokuge, R. E., Blincoe, K., & Bosch, J. (2018, September). Embracing technical debt, from a startup company perspective. In *2018 IEEE international conference on software maintenance and evolution (icsme)* (pp. 415–425). Madrid, Spain: IEEE. doi: 10.1109/ICSME.2018.00053
- Campbell, G. A. (2023, August). *Cognitive complexity: A new way of measuring understandability* (Tech. Rep.). Switzerland: SonarSource S.A. (Version 1.7)
- Cico, O., Souza, R., Jaccheri, L., Nguyen-Duc, A., & Machado, I. (2021). Toward a technical debt relationship with the pivoting of growth phase startups. In *Software business* (Vol. 434, pp. 102–117). Cham: Springer. doi: 10.1007/978-3-030-91452-3\_7
- Cloud Native Computing Foundation. (2022). *CNCF project velocity reports*. <https://www.cncf.io/reports/>. (Refers to the series of project velocity analyses published by the CNCF. Accessed on February 3, 2026)
- Cohn, M. (2005). *Agile estimating and planning*. Prentice Hall.
- Colazo, J. A. (2010, October). Collaboration structure and performance in new software development: Findings from the study of open source projects. *International Journal of Innovation Management*, 14(05), 735–758. doi: 10.1142/s1363919610002866

- Cunningham, W. (1992). The WyCash portfolio management system. In *Addendum to the proceedings of the conference on object-oriented programming systems, languages, and applications (oopsla '92)* (pp. 29–30). New York, NY, USA: ACM. (Also published in ACM SIGPLAN OOPS Messenger, 4(2), Oct. 1992.) doi: 10.1145/157709.157715
- Curtis, B., Sappidi, J., & Szynkarski, A. (2012). Estimating the size, cost, and types of technical debt. In *Proceedings of the 3rd international workshop on managing technical debt (mtd)* (pp. 49–53). IEEE. doi: 10.1109/MTD.2012.6226000
- Cyert, R. M., & March, J. G. (1963). *A behavioral theory of the firm*. Englewood Cliffs, NJ: Prentice-Hall.
- Fisher, I. (1930). *The theory of interest*. New York: The Macmillan Company.
- Fowler, M. (2009, October). *Technical debt quadrant*. <https://martinfowler.com/bliki/TechnicalDebtQuadrant.html>. (Last updated on October 14, 2009.)
- Gompers, P., & Lerner, J. (2001, Spring). The venture capital revolution. *Journal of Economic Perspectives*, 15(2), 145–168. doi: 10.1257/jep.15.2.145
- Gornall, W., & Strebulaev, I. A. (2020). Squaring venture capital valuations with reality. *Journal of Financial Economics*, 135(1), 120–143. doi: 10.1016/j.jfineco.2018.04.015
- Jeklin, U., Saad, M. I., & Ekawati, H. (2025). Evaluation of cocomo model accuracy in software effort estimation. *Bulletin of Information Technology (BIT)*, 6(2), 126–135. (Forthcoming, scheduled for June 2025 issue) doi: 10.47065/bit.v6i2.2027
- Klotins, E., Unterkalmsteiner, M., & Gorschek, T. (2019, February). Software engineering in start-up companies: An analysis of 88 experience reports. *Empirical Software Engineering*, 24(1), 68–102. doi: 10.1007/s10664-018-9620-y
- Kuratko, D. F., Holt, H. L., & Neubert, E. (2020, Jan-Feb). Blitzscaling: The good, the bad, and the ugly. *Business Horizons*, 63(1), 109–119. doi: 10.1016/j.bushor.2019.10.005
- Lee, S. R., & Kim, J. D. (2024). When do startups scale? large-scale evidence from job postings. *Strategic Management Journal*, 45(9), 1633–1669. doi: 10.1002/smj.3537
- Lévesque, M. (2000, February). Effects of funding and its return on product quality in new ventures. *IEEE Transactions on Engineering Management*, 47(1), 98–105. doi: 10.1109/17.820788
- Li, Z., Avgeriou, P., & Liang, P. (2015, March). A systematic mapping study on technical debt and its management. *Journal of Systems and Software*, 101, 193–220. doi: 10.1016/j.jss.2014.12.027

- Lieberman, M. B., & Montgomery, D. B. (1998). First-mover (dis)advantages. *Strategic Management Journal*, 19(12), 1111–1125. doi: 10.1002/(SICI)1097-0266(1998120)19:12<1111::AID-SMJ21>3.0.CO;2-W
- Martini, A., Bosch, J., & Chaudron, M. (2014). Architecture technical debt: Understanding causes and a qualitative model. In *2014 40th euromicro conference on software engineering and advanced applications* (pp. 25–28). doi: 10.1109/SEAA.2014.65
- Morgenthaler, J. D., Gridnev, M., Sauciuc, R., & Bhansali, S. (2012). Searching for build debt: Experiences managing technical debt at Google. In *Proceedings of the 3rd international workshop on managing technical debt (mtd)* (pp. 1–6). IEEE. doi: 10.1109/MTD.2012.6225994
- Nguyen-Duc, A., Khalid, K., Shahid Bajwa, S., & Abrahamsson, P. (2021, October). The entrepreneurial logic of startup software development: A study of 40 software startups. *Empirical Software Engineering*, 26(130). doi: 10.1007/s10664-021-09987-z
- Ning, Y., Wang, W., & Yu, B. (2015, February). The driving forces of venture capital investments. *Small Business Economics*, 44(2), 315–344. doi: 10.1007/s11187-014-9591-3
- Raudenbush, S. W., & Bryk, A. S. (2002). *Hierarchical linear models: Applications and data analysis methods*. Sage Publications.
- Sahlman, W. A. (1990). The structure and governance of venture-capital organizations. *Journal of Financial Economics*, 27(2), 473–521. doi: 10.1016/0304-405X(90)90065-8
- Seaman, C., Guo, Y., Zazworka, N., Shull, F., Izurieta, C., Cai, Y., & Vetrò, A. (2012). Using technical debt data in decision making: Potential decision approaches. In *Proceedings of the 3rd international workshop on managing technical debt (mtd)* (pp. 45–48). IEEE. doi: 10.1109/MTD.2012.6225999
- Simon, H. A. (1955). A behavioral model of rational choice. *The Quarterly Journal of Economics*, 69(1), 99–118.
- Spence, M. (1973). Job market signaling. *The Quarterly Journal of Economics*, 87(3), 355–374.
- Sutherland, J. (2004, September). Agile development: Lessons learned from the first Scrum. *Cutter IT Journal*. (Published online on September 30, 2004.)
- Teece, D. J., Pisano, G., & Shuen, A. (1997). Dynamic capabilities and strategic management. *Strategic Management Journal*, 18(7), 509–533. doi: 10.1002/(SICI)1097-0266(199708)18:7<509::AID-SMJ882>3.0.CO;2-Z



Wang, J., Dou, W., Gao, Y., Gao, C., Qin, F., Yin, K., & Wei, J. (2018). A comprehensive study on real world concurrency bugs in node.js. In *Proceedings of the 33rd acm/ieee international conference on automated software engineering* (pp. 520–531). doi: 10.1145/3238147.3238214

# Appendices

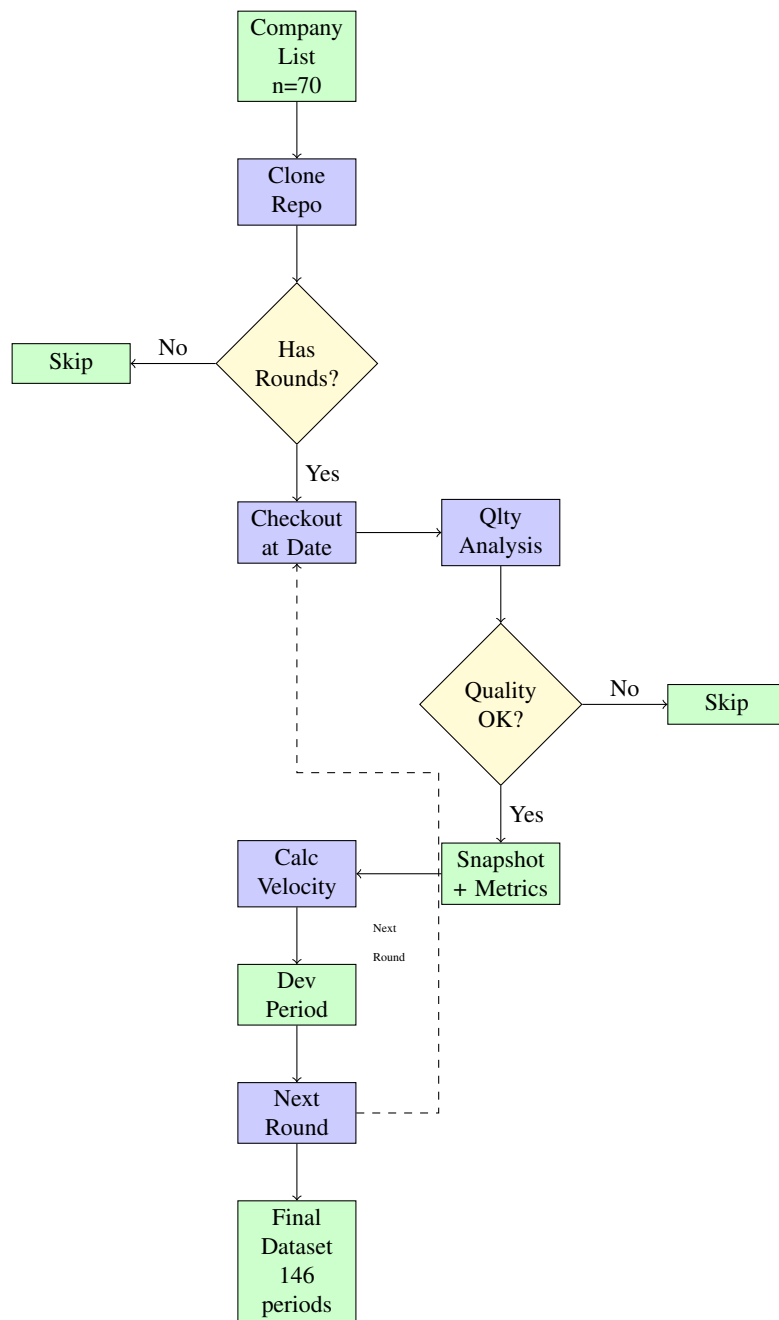


Figure A.1 Automated Analysis Pipeline Architecture

**Source Code:** <https://github.com/maxcodefaster/tdr-velocity-analysis>

Table A.1: Tertile Framework: Success Rates by Detailed Strategic Position

| <b>Strategic Position</b>     | <b>N</b> | <b>Success Rate</b> |
|-------------------------------|----------|---------------------|
| Low Debt + Low Velocity       | 14       | 50.0%               |
| Low Debt + Medium Velocity    | 20       | 40.0%               |
| Low Debt + High Velocity      | 15       | 73.3%               |
| Medium Debt + Low Velocity    | 17       | 52.9%               |
| Medium Debt + Medium Velocity | 18       | 44.4%               |
| Medium Debt + High Velocity   | 14       | 50.0%               |
| High Debt + Low Velocity      | 18       | 44.4%               |
| High Debt + Medium Velocity   | 11       | 72.7%               |
| High Debt + High Velocity     | 19       | 68.4%               |

Table A.2: Venture-Backed Open-Source Software Companies: Funding Timeline Dataset (Part 1 of 3)

| Company        | Category   | GitHub Repository                    | Seed Date | \$M  | Series A Date | \$M  | Series B Date | \$M  | Series C Date | \$M |
|----------------|------------|--------------------------------------|-----------|------|---------------|------|---------------|------|---------------|-----|
| Airbyte        | Data Plat. | github.com/airbytehq/airbyte         | 3/1/20    | 5    | 5/1/21        | 26   | 12/1/21       | 150  | -             | -   |
| Alluxio        | Data Plat. | github.com/Alluxio/alluxio           | -         | -    | 3/17/15       | 7.5  | 1/18/19       | 8.5  | 11/16/21      | 50  |
| Appsmith       | Dev Tools  | github.com/appsmithorg/appsmith      | 3/1/20    | 2.5  | 10/1/21       | 8    | 6/1/22        | 41   | -             | -   |
| Appwrite       | Dev Tools  | github.com/appwrite/appwrite         | 3/1/21    | 10   | 4/1/22        | 27   | -             | -    | -             | -   |
| ArangoDB       | Database   | github.com/arangodb/arangodb         | 2/10/15   | 1.9  | 3/14/19       | 10   | 10/6/21       | 27.8 | -             | -   |
| Balena         | Dev Tools  | github.com/balena-io/etcher          | -         | -    | 4/14/15       | 3    | 7/24/19       | 14.4 | -             | -   |
| Bit            | Dev Tools  | github.com/teambit/bit               | 3/1/15    | 0.9  | 2/1/19        | 10.1 | 11/1/21       | 25   | -             | -   |
| Builder.io     | Bus. App   | github.com/BuilderIO/builder         | 3/1/20    | 3.3  | 10/1/21       | 14   | 4/1/24        | 20   | -             | -   |
| Cloud9 IDE     | Dev Tools  | github.com/c9/core                   | 3/7/11    | 0.75 | 6/1/11        | 5.6  | 11/1/14       | 3.1  | -             | -   |
| Cockroach Labs | Database   | github.com/cockroachdb/cockroach     | 6/3/15    | 6.25 | 3/1/16        | 20   | 6/27/17       | 27   | 8/6/19        | 55  |
| Convex         | Dev Tools  | github.com/get-convex/convex-backend | 11/2/21   | 3.5  | 4/27/22       | 26   | -             | -    | -             | -   |
| Crate.io       | Database   | github.com/crate/crate               | 11/1/15   | 2.8  | 6/1/18        | 11   | -             | -    | -             | -   |
| CrowdSec       | Security   | github.com/crowdsecurity/crowdsec    | 3/1/20    | 2    | 10/1/22       | 14   | -             | -    | -             | -   |
| Dagster        | Data Plat. | github.com/dagster-io/dagster        | 7/12/20   | 1.8  | 11/16/21      | 14   | 3/24/23       | 33   | -             | -   |
| dbt Labs       | Data Plat. | github.com/dbt-labs/dbt-core         | -         | -    | 4/1/20        | 12.9 | 11/1/20       | 29.5 | 6/1/21        | 150 |
| Deepset        | AI/ML      | github.com/deepset-ai/haystack       | 3/8/21    | 1.6  | 4/28/22       | 14   | 8/9/23        | 27   | -             | -   |
| Dgraph         | Database   | github.com/dgraph-io/dgraph          | 3/1/16    | 1.1  | 7/31/19       | 11.5 | -             | -    | -             | -   |
| Directus       | Bus. App   | github.com/directus/directus         | 7/1/21    | 1    | 11/10/22      | 7    | -             | -    | -             | -   |
| Discourse      | Bus. App   | github.com/discourse/discourse       | 11/1/12   | -    | 8/1/21        | 20   | -             | -    | -             | -   |
| EdgeDB         | Database   | github.com/geldata/gel               | 4/1/22    | 4    | 11/1/22       | 15   | -             | -    | -             | -   |
| Elasticsearch  | Database   | github.com/elastic/elasticsearch     | -         | -    | 11/8/12       | 10   | 2/19/13       | 24   | 6/5/14        | 70  |
| Ethica         | Security   | github.com/ethica/fides              | 7/1/19    | 4.2  | 6/11/20       | 13.5 | -             | -    | -             | -   |
| GitLab         | Dev Tools  | github.com/gitlabhq/gitlabhq         | 7/9/15    | 1.5  | 9/15/15       | 4    | 9/27/16       | 20   | 10/10/17      | 20  |
| Gitpod         | Dev Tools  | github.com/gitpod-io/gitpod          | 10/1/20   | 3    | 11/1/22       | 25   | -             | -    | -             | -   |
| Gradle         | Dev Tools  | github.com/gradle/gradle             | 12/1/15   | 4.2  | 5/25/16       | 10.5 | 10/1/18       | 13   | 11/1/21       | 27  |

Table A.3: Venture-Backed Open-Source Software Companies: Funding Timeline Dataset (Part 2 of 3)

| Company        | Category   | GitHub Repository                      | Seed Date | \$M  | Series A Date | \$M  | Series B Date | \$M  | Series C Date | \$M |
|----------------|------------|----------------------------------------|-----------|------|---------------|------|---------------|------|---------------|-----|
| Grafana Labs   | Dev Tools  | github.com/grafana/grafana             | 12/3/18   | 1.2  | 10/24/19      | 24   | 8/17/20       | 50   | 8/24/21       | 220 |
| Harness        | Dev Tools  | github.com/harness/harness             | -         | -    | 10/24/17      | 20   | 4/24/19       | 60   | 1/14/21       | 85  |
| Hasura         | Dev Tools  | github.com/hasura/graphql-engine       | 4/1/18    | 1.6  | 2/26/20       | 9.9  | 9/8/20        | 25   | 2/22/22       | 100 |
| Hazelcast      | Database   | github.com/hazelcast/hazelcast         | -         | -    | 9/19/13       | 2.6  | 9/18/14       | 11   | 6/18/19       | 50  |
| HumanSignal    | AI/ML      | github.com/HumanSignal/label-studio    | 1/1/21    | 4    | 5/1/22        | 25   | -             | -    | -             | -   |
| InfluxData     | Database   | github.com/influxdata/influxdb         | 6/5/13    | 0.8  | 12/8/14       | 8.1  | 9/21/16       | 16   | 2/13/18       | 35  |
| Ionic          | Dev Tools  | github.com/ionic-team/ionic-framework  | 3/1/14    | 1    | 4/1/16        | 8.88 | 2/6/19        | 5.8  | 2/20/20       | 6   |
| Jito Labs      | Dev Tools  | github.com/jito-foundation/jito-solana | 12/21/21  | 2.1  | 8/1/22        | 10   | -             | -    | -             | -   |
| Liquibase      | Database   | github.com/liquibase/liquibase         | 7/18/12   | 0.66 | 12/16/13      | 3    | 1/1/16        | 8    | 2/1/19        | 10  |
| Logical Clocks | AI/ML      | github.com/logicalclocks/hopworks      | 11/1/18   | 1.3  | 7/1/21        | 5    | 6/1/23        | 6.5  | -             | -   |
| Mattermost     | Bus. App   | github.com/mattermost/mattermost       | 2/15/17   | 3.5  | 2/5/19        | 20   | 6/19/19       | 50   | -             | -   |
| Mautic         | Bus. App   | github.com/mautic/mautic               | 6/6/16    | 0.6  | 9/20/16       | 5    | -             | -    | -             | -   |
| MeiliSearch    | Database   | github.com/meilisearch/meilisearch     | 1/27/22   | 5    | 10/10/22      | 15   | -             | -    | -             | -   |
| Metabase       | Data Plat. | github.com/metabase/metabase           | 2/8/19    | 13   | 4/23/19       | 8    | 5/25/21       | 30   | -             | -   |
| MinIO          | Database   | github.com/minio/minio                 | 6/17/15   | 3.3  | 9/19/17       | 20   | 1/26/22       | 103  | -             | -   |
| n8n            | Bus. App   | github.com/n8n-io/n8n                  | 3/1/20    | 1.5  | 4/1/21        | 12   | 3/1/25        | 59.5 | -             | -   |
| Ockam          | Security   | github.com/build-trust/ockam           | 10/15/19  | 4.9  | 1/1/22        | 12.5 | -             | -    | -             | -   |
| Oro            | Bus. App   | github.com/oro-inc/crm-application     | -         | -    | 3/3/16        | 12   | 1/19/23       | 13   | -             | -   |
| Pachyderm      | Data Plat. | github.com/pachyderm/pachyderm         | 6/10/15   | 2    | 11/15/18      | 10   | 8/19/20       | 16   | -             | -   |
| Pimcore        | Bus. App   | github.com/pimcore/pimcore             | -         | -    | 12/1/18       | 3.5  | 9/1/22        | 12   | -             | -   |
| PingCAP        | Database   | github.com/pingcap/tidb                | 4/1/15    | 1.6  | 8/1/16        | 5    | 6/13/17       | 15   | 9/1/18        | 50  |
| Posthog        | Data Plat. | github.com/PostHog/posthog             | 6/4/20    | 3    | 12/17/20      | 9    | 6/10/21       | 15   | 1/1/24        | 10  |
| Prefect        | Data Plat. | github.com/PrefectHQ/prefect           | 12/12/19  | 2.5  | 2/9/21        | 11.5 | 6/10/21       | 32   | -             | -   |
| PrestaShop     | Bus. App   | github.com/PrestaShop/PrestaShop       | -         | -    | 8/1/11        | 3    | 3/1/14        | 9.3  | -             | -   |
| ProcessMaker   | Bus. App   | github.com/ProcessMaker/processmaker   | -         | -    | 2/1/21        | 45   | -             | -    | -             | -   |

Table A.4: Venture-Backed Open-Source Software Companies: Funding Timeline Dataset (Part 3 of 3)

| Company                                                              | Category   | GitHub Repository                               | Seed Date | \$M | Series A Date | \$M  | Series B Date | \$M  | Exit State | Exit Date |
|----------------------------------------------------------------------|------------|-------------------------------------------------|-----------|-----|---------------|------|---------------|------|------------|-----------|
| Reaction Commerce                                                    | Bus. App   | github.com/reactioncommerce/reaction-commerce   | -         | -   | 10/31/17      | 8.5  | -             | -    | Acquired   | 4/22/20   |
| Rocket.Chat                                                          | Bus. App   | github.com/RocketChat/Rocket.Chat               | 10/11/16  | 5   | 2/3/21        | 19   | 9/21/23       | 10   | -          | -         |
| Rudderstack                                                          | Data Plat. | github.com/rudderlabs/rudder-server             | 3/27/20   | 5   | 6/10/21       | 21   | 2/2/22        | 56   | -          | -         |
| Sentry                                                               | Dev Tools  | github.com/getsentry/sentry                     | 8/9/15    | 1.5 | 6/23/16       | 9    | 5/23/18       | 16.5 | -          | -         |
| Serverless                                                           | Dev Tools  | github.com/serverless/serverless                | 10/12/16  | 3   | 7/30/18       | 10   | -             | -    | -          | -         |
| Spree Commerce                                                       | Bus. App   | github.com/spree/spree                          | 7/1/11    | 1.5 | 2/1/14        | 5    | -             | -    | Acquired   | 9/18/15   |
| Starpulse                                                            | Data Plat. | github.com/trinodb/trino                        | -         | -   | 11/1/19       | 22   | 6/1/20        | 42   | -          | -         |
| Strapi                                                               | Bus. App   | github.com/strapi/strapi                        | 10/16/19  | 4   | 5/20/20       | 10   | 6/22/22       | 31   | -          | -         |
| Streamlit                                                            | Data Plat. | github.com/streamlit/streamlit                  | 10/1/19   | 6   | 6/16/20       | 21   | 4/7/21        | 35   | Acquired   | 3/2/22    |
| Supabase                                                             | Dev Tools  | github.com/supabase/supabase                    | 12/15/20  | 6   | 9/9/21        | 30   | 5/10/22       | 80   | -          | -         |
| Teleport                                                             | Security   | github.com/gravitational/teleport               | 5/30/16   | 4.2 | 11/20/19      | 25   | 8/18/21       | 30   | -          | -         |
| Temporal                                                             | Dev Tools  | github.com/temporalio/temporal                  | 11/1/19   | 6.8 | 10/15/20      | 18.8 | 2/16/22       | 103  | -          | -         |
| Tier IV                                                              | AI/ML      | github.com/autowarefoundation/autoware_universe | -         | -   | 7/1/19        | 104  | 7/19/22       | 92   | -          | -         |
| Tigera                                                               | Security   | github.com/tigera/operator                      | -         | -   | 6/1/16        | 13   | 12/1/18       | 30   | -          | -         |
| Timescale                                                            | Database   | github.com/timescale/timescaledb                | 1/1/16    | 3.7 | 1/24/18       | 12.4 | 5/5/21        | 40   | -          | -         |
| Treeverse                                                            | Data Plat. | github.com/treeverse/lakeFS                     | -         | -   | 7/28/21       | 23   | 7/29/25       | 20   | -          | -         |
| Tyk                                                                  | Dev Tools  | github.com/TykTechnologies/tyk                  | -         | -   | 7/1/19        | 5    | 9/1/21        | 35   | -          | -         |
| Unstructured                                                         | AI/ML      | github.com/Unstructured-IO/unstructured         | -         | -   | 7/19/23       | 25   | 3/14/24       | 40   | -          | -         |
| Weaviate                                                             | Database   | github.com/weaviate/weaviate                    | 8/1/20    | 1.2 | 2/1/22        | 16.5 | 4/1/23        | 50   | -          | -         |
| Yugabyte                                                             | Database   | github.com/yugabyte/yugabyte-db                 | 3/1/16    | 8   | 6/1/18        | 16   | 6/1/20        | 30   | -          | -         |
| <b>Dataset Summary:</b>                                              |            |                                                 |           |     |               |      |               |      |            |           |
| Total Companies: 70   Estimated Development Periods: 120-160         |            |                                                 |           |     |               |      |               |      |            |           |
| Temporal Coverage: 2009-2025 (ZIRP era: 2009-2022)                   |            |                                                 |           |     |               |      |               |      |            |           |
| Exit Events: 6 acquisitions, 2 IPOs (GitLab, Elasticsearch)          |            |                                                 |           |     |               |      |               |      |            |           |
| All GitHub repositories verified as active primary product codebases |            |                                                 |           |     |               |      |               |      |            |           |

### **A.0.1 Data Collection Notes**

All funding information was collected from Crunchbase and Tracxn databases and cross-validated across multiple sources. GitHub repository links are verified as active and pointing to the primary product codebase for each company. The GitHub URLs listed use abbreviated format (without https://) for space efficiency; the automated analysis pipeline will use full URLs for repository cloning.

Series D, E, F, and G funding rounds are documented in the complete dataset but excluded from these summary tables for space constraints. The dataset represents a comprehensive sample of venture-backed open-source software companies that raised Series A funding during or after 2009, with at least 36 months elapsed between Series A and data collection to allow for subsequent funding evaluation.